

Computer Architecture

CS 622 / EE 520

Spring 2026

Lecture 10
Shahid Masud

- Loop Unrolling
- Dynamic Scheduling
- Introducing Tomasulo Architecture

**Quiz 2
TODAY**

Reading Notes

2019 EE 488

Dynamic Scheduling - Scoreboard -

Myoungsoo Jung
Computer Division

Readings:
Chapter 3 of course textbook

Computer Architecture and Memory systems Laboratory

KAIST EE

CAMELab



Recap: Instruction Level Parallelism

Starting Chapter 3 of textbook

How to go about ILP



- Pipelining become universal technique in 1985
 - Overlaps execution of instructions
 - Exploits “Instruction Level Parallelism”
- Beyond this, there are two main approaches:
 - Hardware-based dynamic approaches
 - Used in server and desktop processors
 - Not used as extensively in IoT, Low Power, processors
 - Compiler-based static approaches
 - Not as successful outside of scientific applications

What Is Instruction-Level Parallelism?

All the techniques in this chapter exploit parallelism among instructions. The amount of parallelism available within a *basic block*—a straight-line code sequence with no branches in except to the entry and no branches out except at the exit—is quite small. For typical RISC programs, the average dynamic branch frequency is often between 15% and 25%, meaning that between three and six instructions execute between a pair of branches. Because these instructions are likely to depend upon one another, the amount of overlap we can exploit within a basic block is likely to be less than the average basic block size. To obtain substantial performance enhancements, we must exploit ILP across multiple basic blocks.

- Dependencies are a property of programs
- Pipeline organization determines if dependence is detected and if it causes a stall
- Data dependence conveys:
 - Possibility of a hazard
 - Order in which results must be calculated
 - Upper bound on exploitable instruction level parallelism
- Dependencies that flow through memory locations are difficult to detect

Overcome Dependence



Because a data dependence can limit the amount of instruction-level parallelism we can exploit, a major focus of this chapter is overcoming these limitations. A dependence can be overcome in two different ways: (1) maintaining the dependence but avoiding a hazard, and (2) eliminating a dependence by transforming the code. Scheduling the code is the primary method used to avoid a hazard without altering a dependence, and such scheduling can be done both by the compiler and by the hardware.

Dependence types – code examples



- RAW (Read After Write) = "true dependence"

mul r0 * r1 -> **r2**

...

add **r2** + r3 -> r4

- WAW (Write After Write) = "output dependence"

mul r0 * r1 -> **r2**

...

add r1 + r3 -> **r2**

- WAR (Write After Read) = "anti-dependence"

mul r0 * **r1** -> r2

...

add r3 + r4 -> **r1**

- WAW & WAR are "false", Can be totally eliminated by "renaming"

Dependence Chain

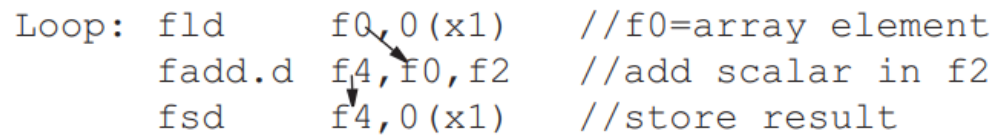
This dependence chain can be as long as the entire program. Note that a dependence within a single instruction (such as `add x1, x1, x1`) is not considered a dependence.

For example, consider the following RISC-V code sequence that increments a vector of values in memory (starting at `0(x1)` ending with the last element at `0(x2)`) by a scalar in register `f2`.

```
Loop: fld      f0,0(x1)    //f0=array element
      fadd.d   f4,f0,f2    //add scalar in f2
      fsd      f4,0(x1)    //store result
      addi     x1,x1,-8     //decrement pointer 8 bytes
      bne      x1,x2,Loop  //branch x1≠x2
```

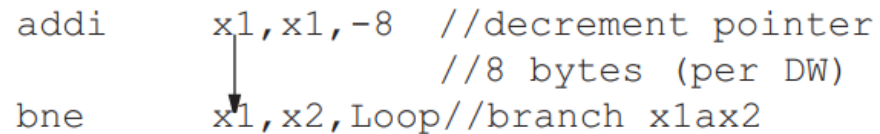
The data dependences in this code sequence involve both floating-point data:

```
Loop: fld      f0,0(x1)    //f0=array element
      fadd.d   f4,f0,f2    //add scalar in f2
      fsd      f4,0(x1)    //store result
```



and integer data:

```
addi     x1,x1,-8    //decrement pointer
                          //8 bytes (per DW)
bne      x1,x2,Loop  //branch x1≠x2
```



In both of the preceding dependent sequences, as shown by the arrows, each instruction depends on the previous one. The arrows here and in following examples show the order that must be preserved for correct execution. The arrow points from an instruction that must precede the instruction that the arrowhead points to.

Name Dependence and Output Dependence

- An **Antidependence** between instruction i and instruction j occurs when instruction j writes a register or memory that instruction i reads.
 - The original ordering must be preserved to ensure that i reads the correct value. In example, there is antidependence between fsd and addi on register x1.
- An **Output dependence** occurs when instruction i and instruction j write the same register or memory location.
 - The ordering between the instruction must be preserved to ensure that value finally written corresponds to instruction j.

• Both Antidependence and Output Dependence are **Name Dependences**, as opposed to True Dependences, as no value is being transmitted between the two instructions.

```

Loop: fld      f0,0(x1)    //f0=array element
      fadd.d   f4,f0,f2    //add scalar in f2
      fsd      f4,0(x1)    //store result
      addi     x1,x1,-8     //decrement pointer 8 bytes
      bne      x1,x2,Loop  //branch x1≠x2
    
```

The data dependences in this code sequence involve both floating-point data:

```

Loop: fld      f0,0(x1)    //f0=array element
      fadd.d   f4,f0,f2    //add scalar in f2
      fsd      f4,0(x1)    //store result
    
```

and integer data:

```

      addi     x1,x1,-8     //decrement pointer
                          //8 bytes (per DW)
      bne      x1,x2,Loop  //branch x1≠x2
    
```

More on Data Dependences

Dependences are a property of *programs*. Whether a given dependence results in an actual hazard being detected and whether that hazard actually causes a stall are properties of the *pipeline organization*. This difference is critical to understanding how instruction-level parallelism can be exploited.

A data dependence conveys three things: (1) the possibility of a hazard, (2) the order in which results must be calculated, and (3) an upper bound on how much parallelism can possibly be exploited. Such limits are explored in a pitfall on [page 262](#) and in Appendix H in more detail.

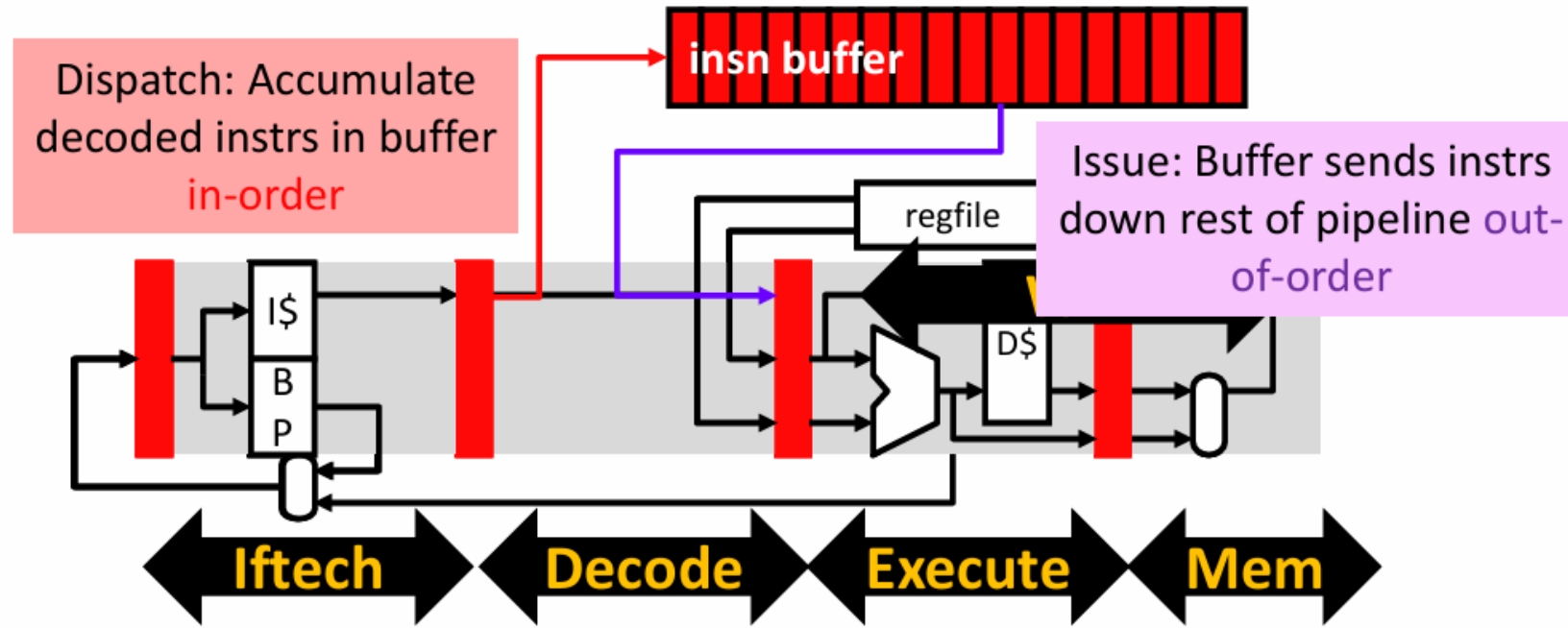
Because a data dependence can limit the amount of instruction-level parallelism we can exploit, a major focus of this chapter is overcoming these limitations. A dependence can be overcome in two different ways: (1) maintaining the dependence but avoiding a hazard, and (2) eliminating a dependence by transforming the code. Scheduling the code is the primary method used to avoid a hazard without altering a dependence, and such scheduling can be done both by the compiler and by the hardware.

A data value may flow between instructions either through registers or through memory locations. When the data flow occurs through a register, detecting the dependence is straightforward because the register names are fixed in the instructions, although it gets more complicated when branches intervene and correctness concerns force a compiler or hardware to be conservative.

Scoreboard Scheduling Technique

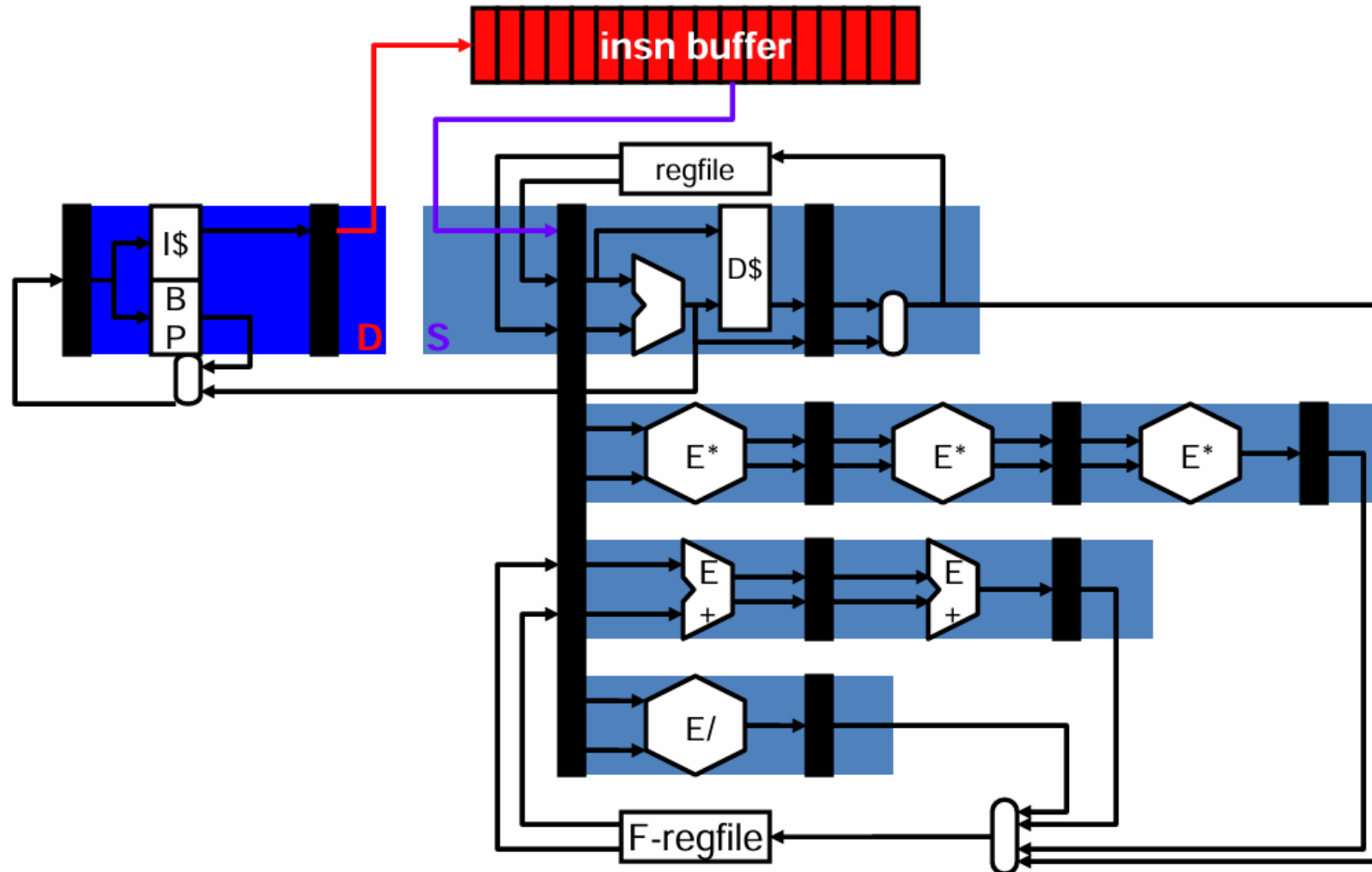
Scoreboard Hardware

The key idea of Scoreboard



- Conventional 5 pipeline stages
Iftech → Decode → Execute → Memory → Writeback
- Let's split Decode stage into two sub stages
Dispatch (D) & Issue (S)

Dispatch and Issue with Floating-Point

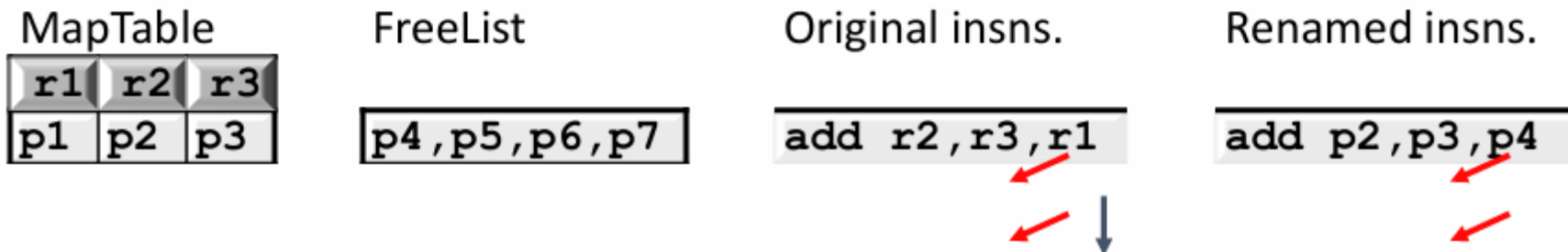


Register Renaming

- Register renaming (in hardware)
 - “Change” register names to eliminate WAR/WAW hazards
 - Arch. registers (r1,f0...) are *names*, not storage locations
 - Can have more locations than names
 - Can have multiple active versions of same name
- How does it work?
 - **Map-table**: maps names to most recent locations
 - On a write: allocate new location (from a **free list**), note in map-table
 - On a read: find location of most recent write via map-table

Register Renaming

- Anti (**WAR**) and output (**WAW**) deps. are false
 - Dep. is on name/location, not on data
 - Given infinite registers, WAR/WAW don't arise
 - Renaming removes WAR/WAW, but leaves **RAW** intact
- Example
 - Names: r1,r2,r3 Physical Locations: p1–p7
 - Original: r1→p1, r2→p2, r3→p3, p4–p7 are “free”



Register Renaming

- Anti (**WAR**) and output (**WAW**) deps. are false
 - Dep. is on name/location, not on data
 - Given infinite registers, WAR/WAW don't arise
 - Renaming removes WAR/WAW, but leaves **RAW** intact
- Example
 - Names: r1,r2,r3 Physical Locations: p1–p7
 - Original: r1→p1, r2→p2, r3→p3, p4–p7 are “free”

MapTable	FreeList	Original insns.	Renamed insns.																																			
<table><tr><th>r1</th><th>r2</th><th>r3</th></tr><tr><td>p1</td><td>p2</td><td>p3</td></tr><tr><td>p4</td><td>p2</td><td>p3</td></tr><tr><td>p4</td><td>p2</td><td>p5</td></tr><tr><td>p4</td><td>p2</td><td>p6</td></tr></table>	r1	r2	r3	p1	p2	p3	p4	p2	p3	p4	p2	p5	p4	p2	p6	<table><tr><td>p4 , p5 , p6 , p7</td></tr><tr><td>p5 , p6 , p7</td></tr><tr><td>p6 , p7</td></tr><tr><td>p7</td></tr></table>	p4 , p5 , p6 , p7	p5 , p6 , p7	p6 , p7	p7	<table><tr><td>add</td><td>r2 , r3 , r1</td></tr><tr><td>sub</td><td>r2 , r1 , r3</td></tr><tr><td>mul</td><td>r2 , r3 , r3</td></tr><tr><td>div</td><td>r1 , 4 , r1</td></tr></table>	add	r2 , r3 , r1	sub	r2 , r1 , r3	mul	r2 , r3 , r3	div	r1 , 4 , r1	<table><tr><td>add</td><td>p2 , p3 , p4</td></tr><tr><td>sub</td><td>p2 , p4 , p5</td></tr><tr><td>mul</td><td>p2 , p5 , p6</td></tr><tr><td>div</td><td>p4 , 4 , p7</td></tr></table>	add	p2 , p3 , p4	sub	p2 , p4 , p5	mul	p2 , p5 , p6	div	p4 , 4 , p7
r1	r2	r3																																				
p1	p2	p3																																				
p4	p2	p3																																				
p4	p2	p5																																				
p4	p2	p6																																				
p4 , p5 , p6 , p7																																						
p5 , p6 , p7																																						
p6 , p7																																						
p7																																						
add	r2 , r3 , r1																																					
sub	r2 , r1 , r3																																					
mul	r2 , r3 , r3																																					
div	r1 , 4 , r1																																					
add	p2 , p3 , p4																																					
sub	p2 , p4 , p5																																					
mul	p2 , p5 , p6																																					
div	p4 , 4 , p7																																					

Register Renaming Example

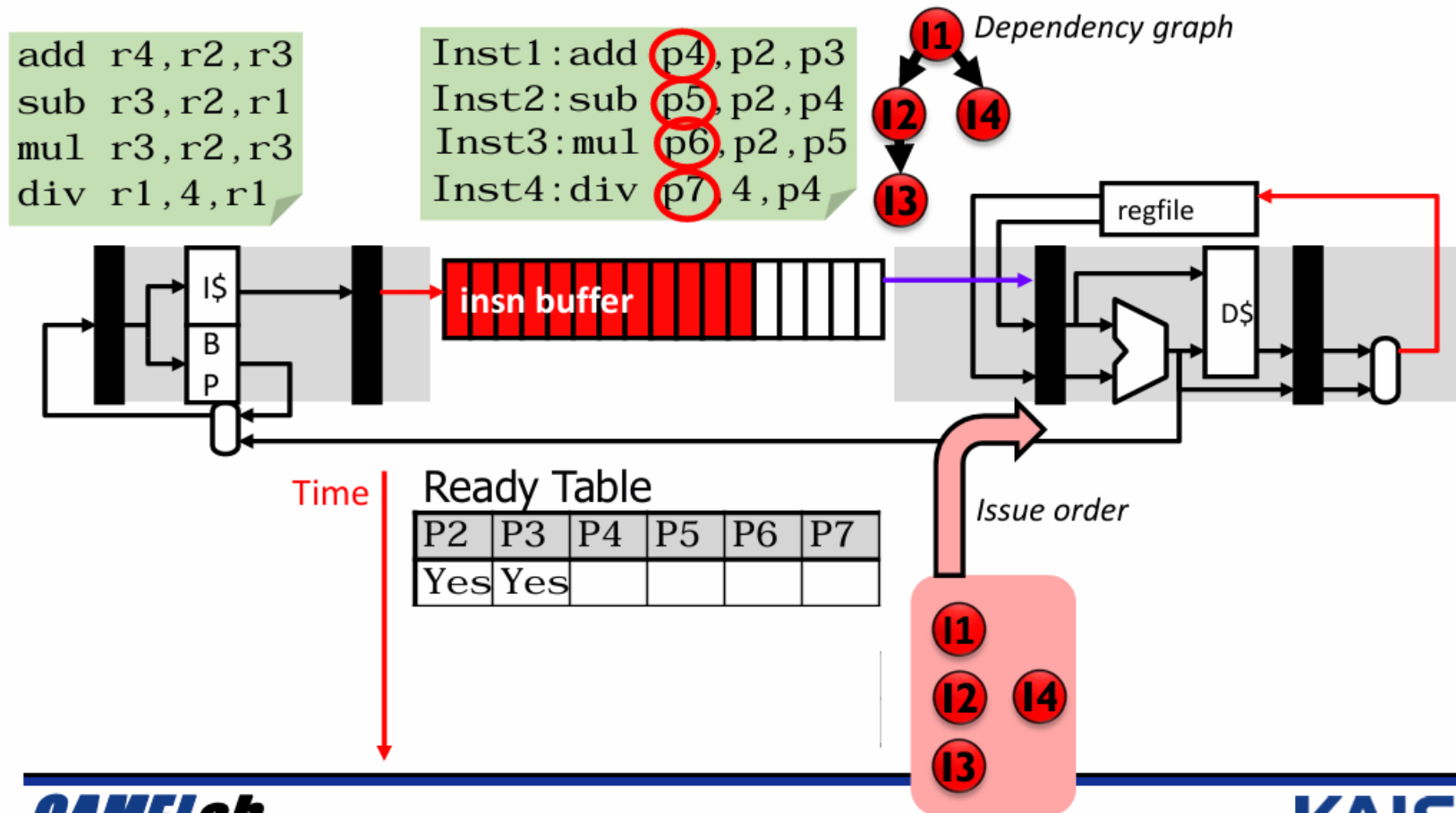
- names: r1, r2, r3, locations: 11, 12, 13, 14, 15, 16, 17
- original mapping: r1→11, r2→12, r3→13 (14-17 "free")

raw instructions	map table	free locations	renamed instructions																		
	<table><tr><th>r1</th><th>r2</th><th>r3</th></tr><tr><td>11</td><td>12</td><td>13</td></tr><tr><td>14</td><td>12</td><td>13</td></tr><tr><td>14</td><td>12</td><td>15</td></tr><tr><td>16</td><td>12</td><td>15</td></tr><tr><td>16</td><td>17</td><td>15</td></tr></table>	r1	r2	r3	11	12	13	14	12	13	14	12	15	16	12	15	16	17	15	14, 15, 16, 17	
r1	r2	r3																			
11	12	13																			
14	12	13																			
14	12	15																			
16	12	15																			
16	17	15																			
add r1, r2, r3		15, 16, 17	add 14, 12, 13																		
sub r3, r2, r1		16, 17	sub 15, 12, 14																		
mul r1, r2, r3		17	mul 16, 12, 15																		
div r2, r1, r3			div 17, 16, 15																		

- renaming removes WAW/WAR, leaves RAW intact!!

Dynamic Scheduling: The Big Picture

- Instructions fetch/decoded/renamed into *Instruction Buffer*
- Instructions (conceptually) check ready bits every cycle



Simple Scoreboard Data Structures

Register Result Status

➤ Three main components

- Instruction status
- Functional unit status
- Register result status

- Indicates which functional unit will **write** each register, if one exists.
- Blank when no pending instructions will write that register

▶ Instruction Status

- Which of 4 steps the instruction is in
 - D: Issue
 - S: Read operands
 - X: Execute stage completion
 - W: Write result to registers

▶ Functional Unit Status

- Indicates the state of the functional unit (FU)
- 9 fields for each FU

B:	Indicates whether the unit is busy or not
Op:	Operation to perform in the unit (e.g., + or -)
dst:	Destination register
src1,src2:	Source-register numbers
Q1, Q2:	Functional units producing source registers src1, src2
R1, R2:	Flags being set when src1/src2 is ready

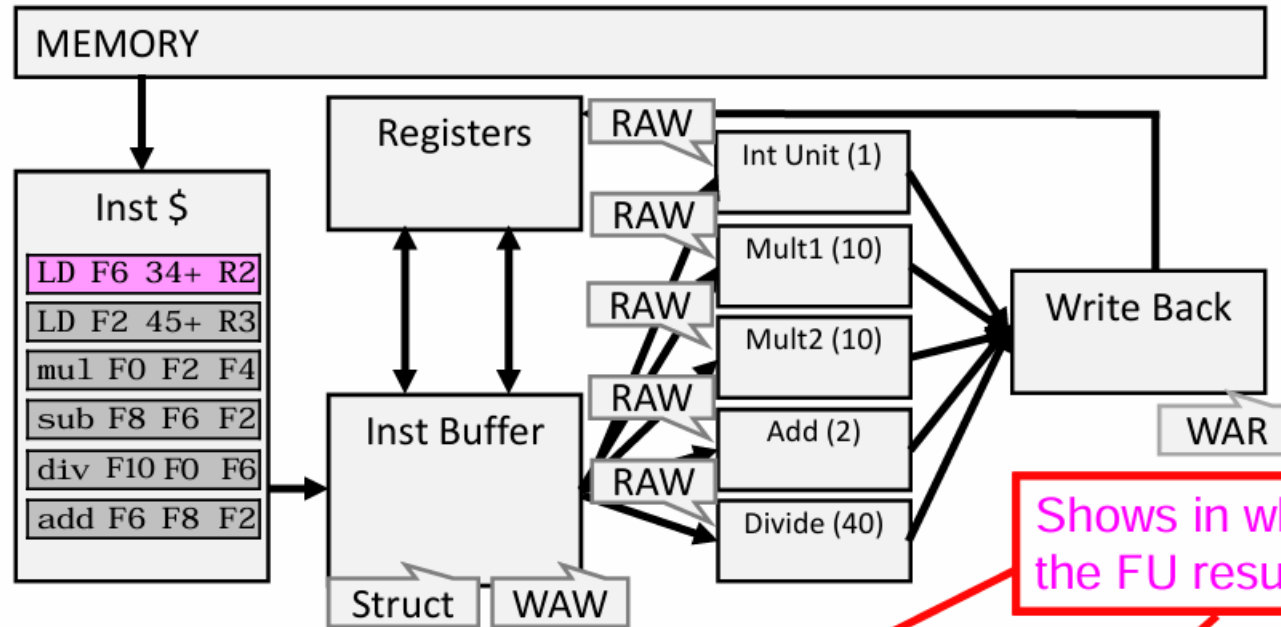
Example Scoreboard

Scoreboard

Insn Status								FU Status										Reg Status	
Inst	dst	src1	src2	D	S	X	W	FU	B	Op	dst	src1	src2	Q1	Q2	R1	R2		FU
LD	F6	34+	R2					Int										F0	
LD	F2	45+	R3					Mult1										F2	
MULTD	F0	F2	F4					Mult2										F4	
SUBD	F8	F6	F2					Add										F6	
DIVD	F10	F0	F6					Div										F8	
ADDD	F6	F8	F2															F10	
																		...	

Scoreboard Cycle 1

➤ Scoreboard Example: Cycle 1



Try to Issue LD #1

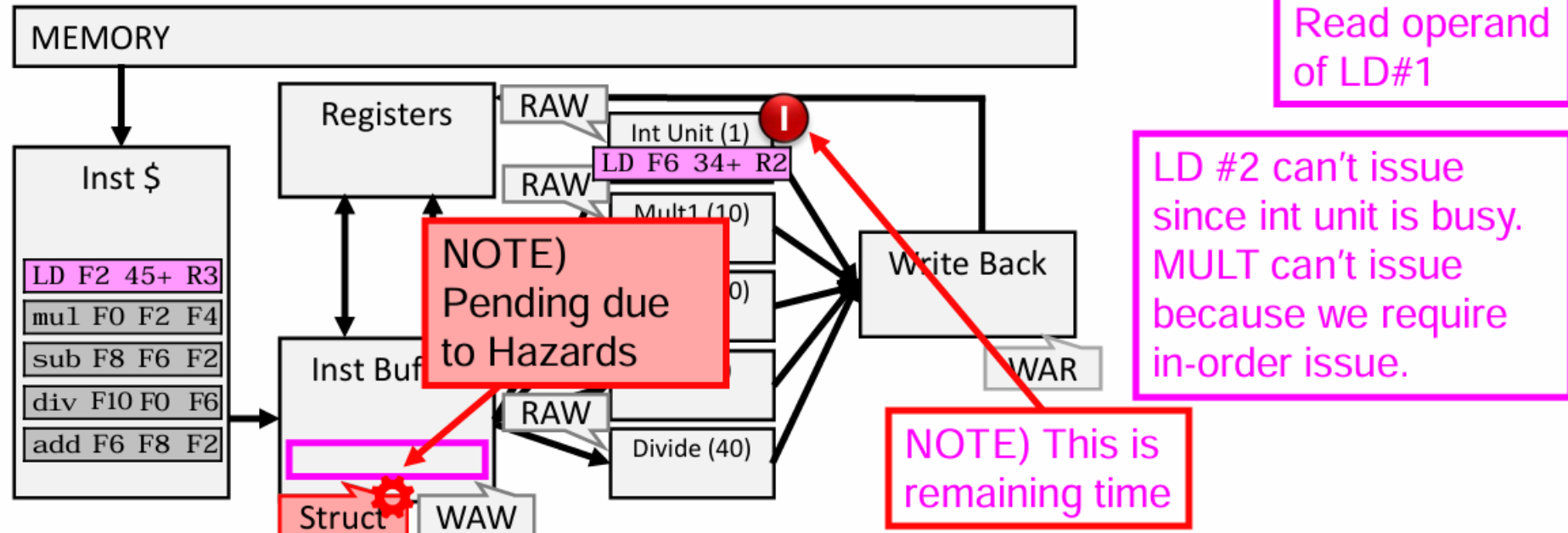
Check Struct & WAW Hazards
→ No Hazards!

Shows in which register the FU result is updated

Scoreboard															
Insn Status								FU Status							
Inst	dst	src1	src2	D	S	X	W	FU	B	Op	dst	src1	src2	Q1	Q2
LD	F6	34+	R2	1				Int	Yes	LD	F6		R2		
LD	F2	45+	R3					Mult1	No						
MULTD	F0	F2	F4					Mult2	No						
SUBD	F8	F6	F2					Add	No						
DIVD	F10	F0	F6					Div	No						
ADD	F6	F8	F2												

Reg Status	
Reg	FU
F0	
F2	
F4	
F6	Int
F8	
F10	
...	

Scoreboard Example: Cycle 2



Scoreboard

Insn Status							
Inst	dst	src1	src2	D	S	X	W
LD	F6	34+	R2	1	2		
LD	F2	45+	R3				
MULTD	F0	F2	F4				
SUBD	F8	F6	F2				
DIVD	F10	F0	F6				
ADD	F6	F8	F2				

FU Status									
FU	B	Op	dst	src1	src2	Q1	Q2	R1	R2
Int	Yes	LD	F6		R2				yes
Mult1	No								
Mult2	No								
Add	No								
Div	No								

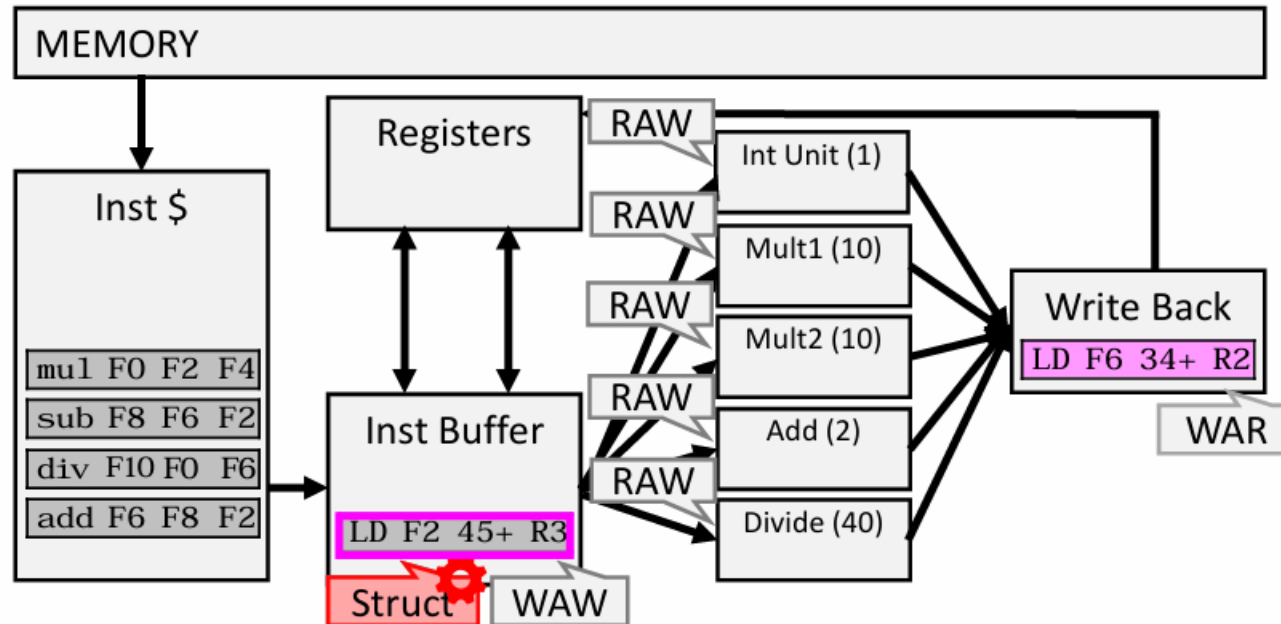
Reg Status	
	FU
F0	
F2	
F4	
F6	Int
F8	
F10	
...	



Reg Status	
	FU
F0	
F2	
F4	
F6	Int
F8	
F10	
...	

Scoreboard Cycle 4

➤ Scoreboard Example: Cycle 4



WB LD#1

Free FU & Reg
status of LD#1

Integer unit free
→ No Hazard!

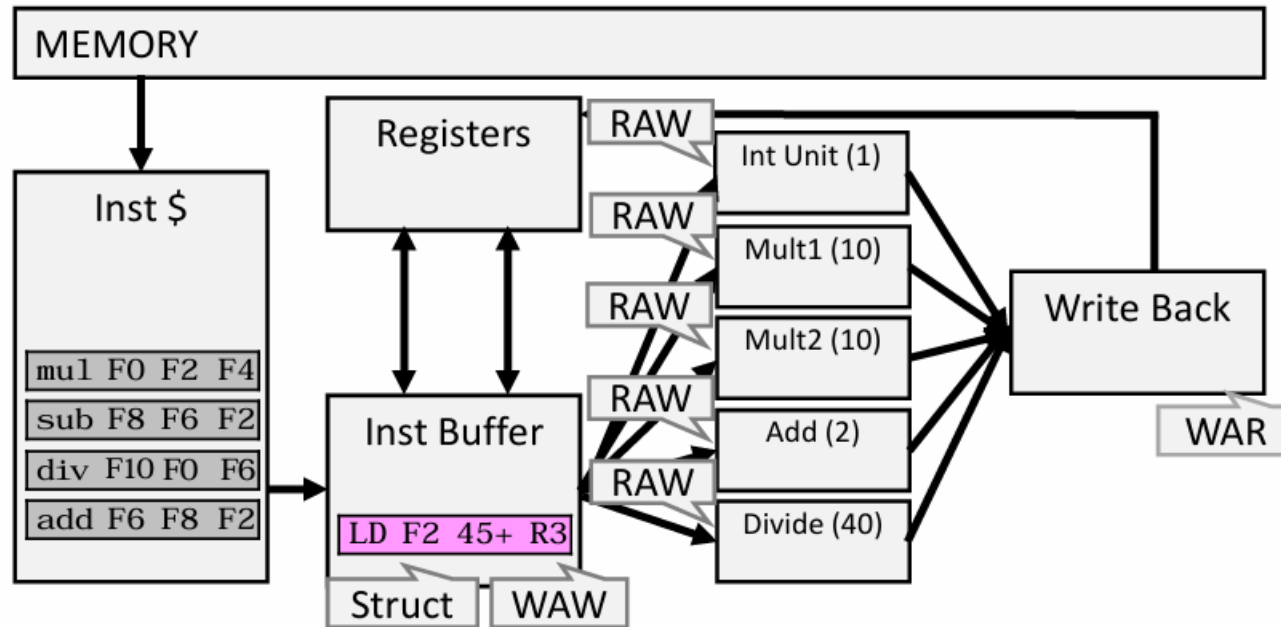
Scoreboard

Insn Status							
Inst	dst	src1	src2	D	S	X	W
LD	F6	34+	R2	1	2	3	4
LD	F2	45+	R3				
MULTD	F0	F2	F4				
SUBD	F8	F6	F2				
DIVD	F10	F0	F6				
ADDD	F6	F8	F2				

FU Status									
FU	B	Op	dst	src1	src2	Q1	Q2	R1	R2
Int	No								
Mult1	No								
Mult2	No								
Add	No								
Div	No								

Reg Status	
	FU
F0	
F2	
F4	
F6	
F8	
F10	
...	

Issue LD #2 since
int unit is now free.



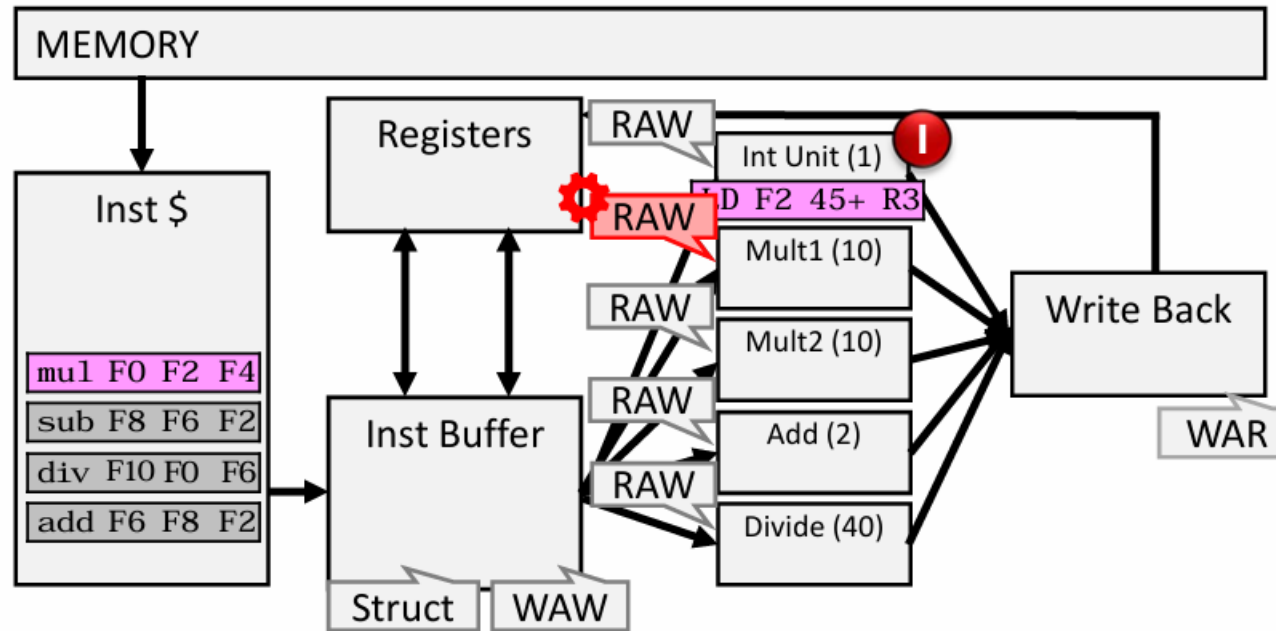
Scoreboard							
Insn Status							
Inst	dst	src1	src2	D	S	X	W
LD	F6	34+	R2	1	2	3	4
LD	F2	45+	R3	5			
MULTD	F0	F2	F4				
SUBD	F8	F6	F2				
DIVD	F10	F0	F6				
ADDD	F6	F8	F2				

FU Status									
FU	B	Op	dst	src1	src2	Q1	Q2	R1	R2
Int	Yes	LD	F2		R3				yes
Mult1	No								
Mult2	No								
Add	No								
Div	No								

Reg Status	
	FU
F0	
F2	Int
F4	
F6	
F8	
F10	
...	

Scoreboard Cycle 6

➤ Scoreboard Example: Cycle 6



Read operand
of LD#2

Try to Issue MULT

Check Hazards
→ No Hazards!

NOTE) There is
RAW due to LD#2

Scoreboard

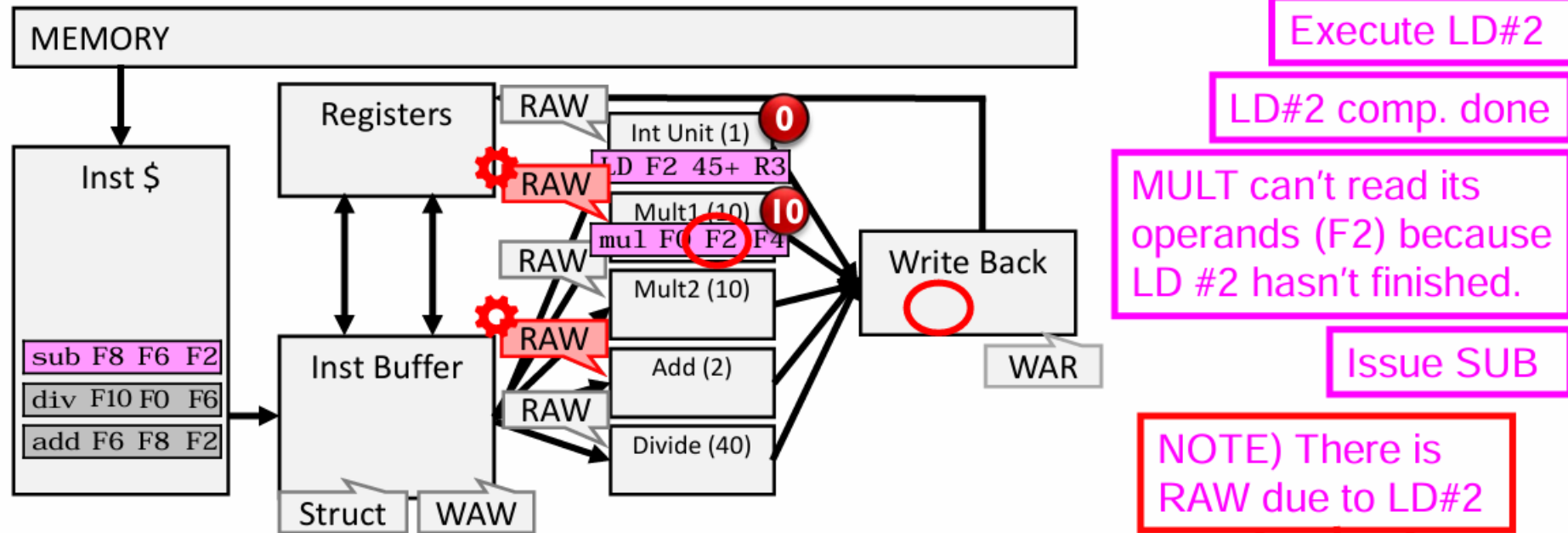
Instn Status							
Inst	dst	src1	src2	D	S	X	W
LD	F6	34+	R2	1	2	3	4
LD	F2	45+	R3	5	6		
MULTD	F0	F2	F4	6			
SUBD	F8	F6	F2				
DIVD	F10	F0	F6				
ADD	F6	F8	F2				

FU Status									
FU	B	Op	dst	src1	src2	Q1	Q2	R1	R2
Int	Yes	LD	F2		R3				yes
Mult1	Yes	MUL	F0	F2	F4	Int		No	yes
Mult2	No								
Add	No								
Div	No								

Reg Status	
Reg	FU
F0	Mult1
F2	Int
F4	
F6	
F8	
F10	
...	

Scoreboard Cycle 7

➤ Scoreboard Example: Cycle 7



Scoreboard							
Insn Status							
Inst	dst	src1	src2	D	S	X	W
LD	F6	34+	R2	1	2	3	4
LD	F2	45+	R3	5	6	7	
MULTD	F0	F2	F4	6			
SUBD	F8	F6	F2	7			
DIVD	F10	F0	F6				
ADDD	F6	F8	F2				

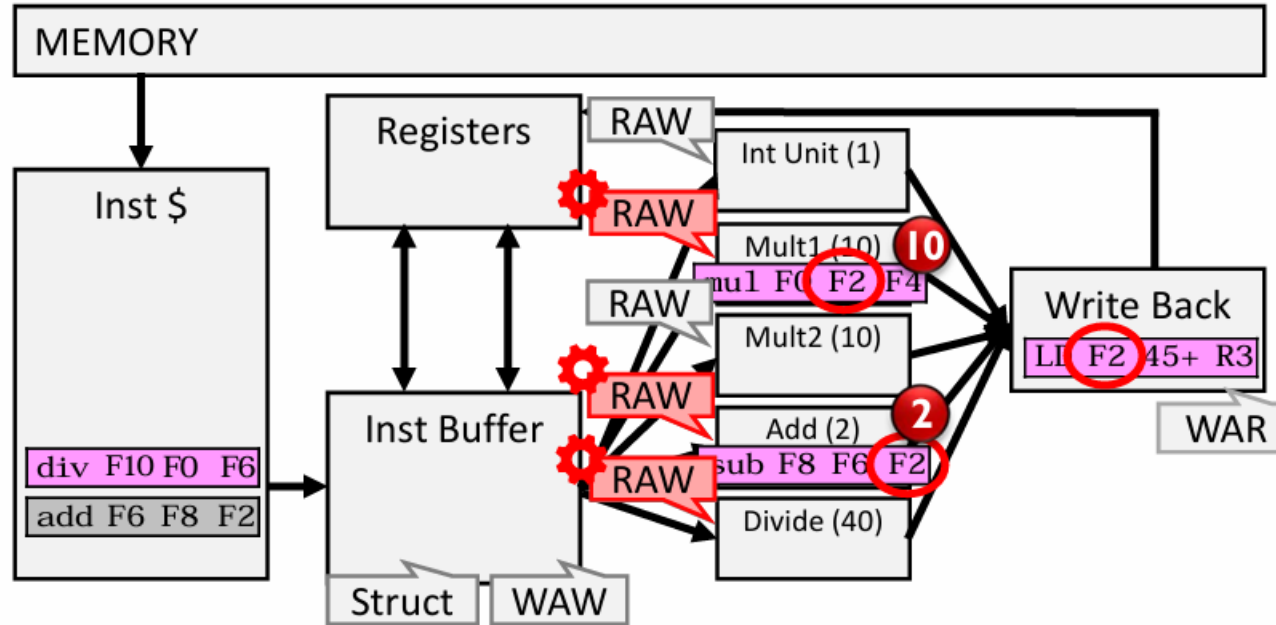
FU Status									
FU	B	Op	dst	src1	src2	Q1	Q2	R1	R2
Int	Yes	LD	F2		R3				yes
Mult1	Yes	MUL	F0	F2	F4	Int		No	yes
Mult2	No								
Add	Yes	SUB	F8	F6	F2		Int	Yes	No
Div	No								

Reg Status	
Reg	FU
F0	Mult1
F2	Int
F4	
F6	
F8	Add
F10	
...	

Scoreboard Cycle 8a

➤ Scoreboard Example: Cycle 8a

First half of clock cycle



Issue DIVD

MULT & SUBD both waiting for F2

NOTE) There is RAW due to Mult1

Scoreboard

Instn Status							
Inst	dst	src1	src2	D	S	X	W
LD	F6	34+	R2	1	2	3	4
LD	F2	45+	R3	5	6	7	
MULTD	F0	F2	F4	6			
SUBD	F8	F6	F2	7			
DIVD	F10	F0	F6	8			
ADD	F6	F8	F2				

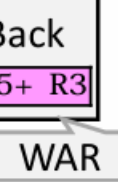
FU Status									
FU	B	Op	dst	src1	src2	Q1	Q2	R1	R2
Int	Yes	LD	F2		R3				yes
Mult1	Yes	MUL	F0	F2	F4	Int		No	yes
Mult2	No								
Add	Yes	SUB	F8	F6	F2		Int	Yes	No
Div	Yes	DIV	F10	F0	F6	Mult1		No	Yes

Reg Status	
Reg	FU
F0	Mult1
F2	Int
F4	
F6	
F8	Add
F10	Div
...	

LD#2 writes F2

Free FU & Reg
status of LD#2

Now, F2 is ready



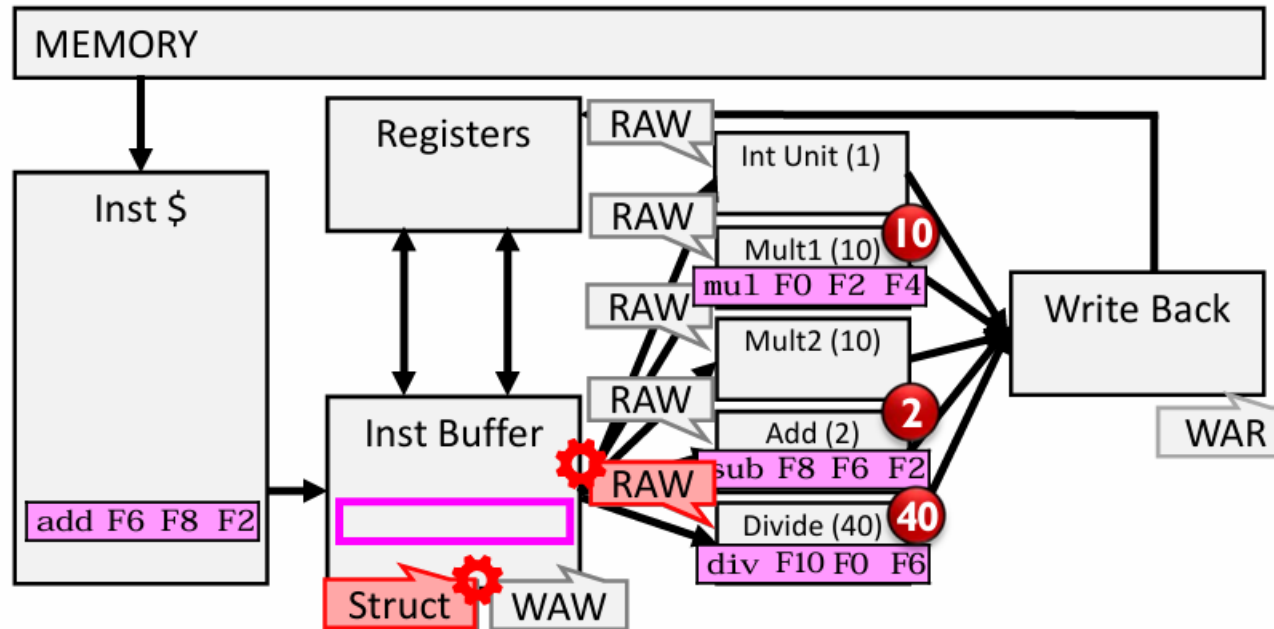
Insn Status							
Inst	dst	src1	src2	D	S	X	W
LD	F6	34+	R2	1	2	3	4
LD	F2	45+	R3	5	6	7	8
MULTD	F0	F2	F4	6			
SUBD	F8	F6	F2	7			
DIVD	F10	F0	F6	8			
ADDD	F6	F8	F2				

FU Status									
FU	B	Op	dst	src1	src2	Q1	Q2	R1	R2
Int	No								
Mult1	Yes	MUL	F0	F2	F4			Yes	yes
Mult2	No								
Add	Yes	SUB	F8	F6	F2			Yes	Yes
Div	Yes	DIV	F10	F0	F6	Mul1		No	Yes

	FU
--	----

Scoreboard Cycle 9

➤ Scoreboard Example: Cycle 9



Try to Issue MULT

ADDD cant start
∴ add unit is busy

Now MULT and SUBD
can both read F2.

Scoreboard

Insn Status							
Inst	dst	src1	src2	D	S	X	W
LD	F6	34+	R2	1	2	3	4
LD	F2	45+	R3	5	6	7	8
MULTD	F0	F2	F4	6	9		
SUBD	F8	F6	F2	7	9		
DIVD	F10	F0	F6	8			
ADDD	F6	F8	F2				

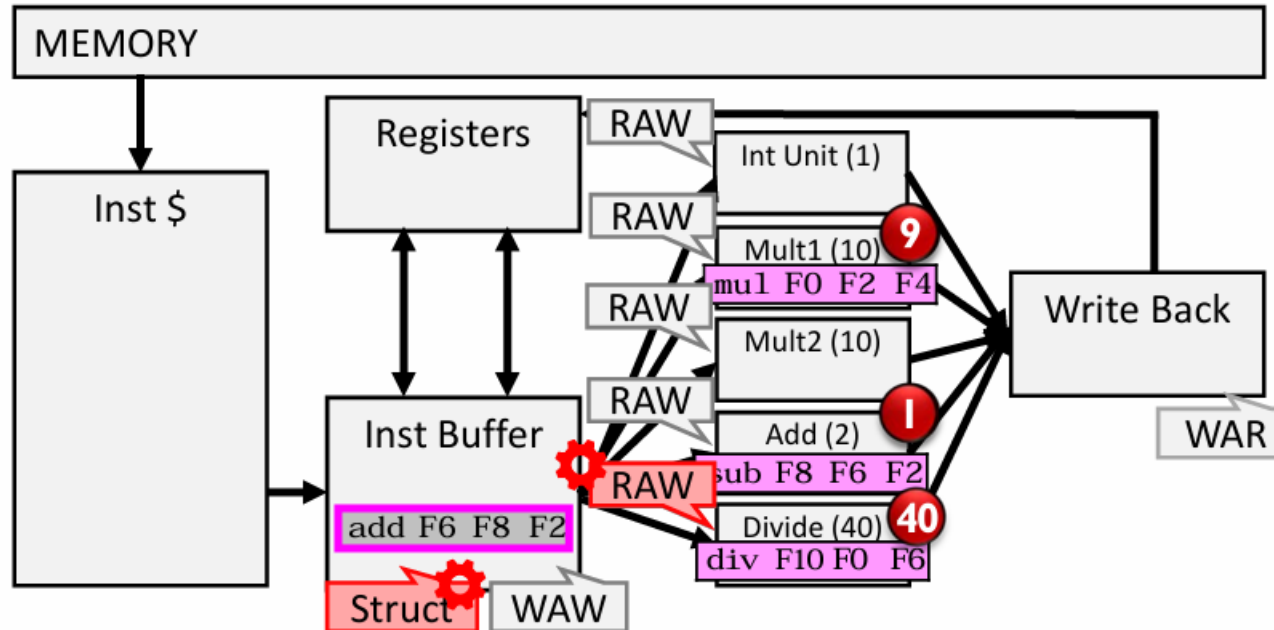
FU Status									
FU	B	Op	dst	src1	src2	Q1	Q2	R1	R2
Int	No								
Mult1	Yes	MUL	F0	F2	F4			Yes	yes
Mult2	No								
Add	Yes	SUB	F8	F6	F2			Yes	Yes
Div	Yes	DIV	F10	F0	F6	Mult1		No	Yes

Reg Status	
	FU
F0	Mult1
F2	
F4	
F6	
F8	Add
F10	Div
...	

Scoreboard Cycle 10

➤ Scoreboard Example: Cycle 10

Calculating....



Scoreboard

Instn Status							
Inst	dst	src1	src2	D	S	X	W
LD	F6	34+	R2	1	2	3	4
LD	F2	45+	R3	5	6	7	8
MULTD	F0	F2	F4	6	9		
SUBD	F8	F6	F2	7	9		
DIVD	F10	F0	F6	8			
ADDD	F6	F8	F2				

FU Status									
FU	B	Op	dst	src1	src2	Q1	Q2	R1	R2
Int	No								
Mult1	Yes	MUL	F0	F2	F4			Yes	yes
Mult2	No								
Add	Yes	SUB	F8	F6	F2			Yes	Yes
Div	Yes	DIV	F10	F0	F6	Mult1		No	Yes

Reg Status	
	FU
F0	Mult1
F2	
F4	
F6	
F8	Add
F10	Div
...	

2019 EE 488

Dynamic Scheduling - Scoreboard -

Myoungsoo Jung
Computer Division

Computer Architecture and Memory systems Laboratory

KAIST EE

CAMELab



Loop Unrolling ILP

Basic Compiler Techniques for Exposing ILP

This section examines the use of simple **compiler technology to enhance a processor's ability to exploit ILP. These techniques are crucial for processors that use static issue or static scheduling.** Armed with this compiler technology, we will shortly examine the design and performance of processors using static issuing. Appendix H will investigate more sophisticated compiler and associated hardware schemes designed to enable a processor to exploit more instruction-level parallelism.

Basic Pipeline Scheduling and Loop Unrolling

To keep a pipeline full, parallelism among instructions must be exploited by finding sequences of unrelated instructions that can be overlapped in the pipeline. To **avoid a pipeline stall, the execution of a dependent instruction must be separated from the source instruction by a distance in clock cycles equal to the pipeline latency of that source instruction.** A compiler's ability to perform this scheduling depends both on the amount of ILP available in the program and on the latencies of the functional units in the pipeline. **Figure 3.2** shows the FP unit latencies we

Summary of the Loop Unrolling and Scheduling

Throughout this chapter and Appendix H, we will look at a variety of hardware and software techniques that allow us to take advantage of instruction-level parallelism to fully utilize the potential of the functional units in a processor. The key to most of these techniques is to know when and how the ordering among instructions may be changed. In our example, we made many such changes, which to us, as human beings, were obviously allowable. In practice, this process must be performed in a methodical fashion either by a compiler or by hardware. To obtain the final unrolled code, we had to make the following decisions and transformations:

- Determine that unrolling the loop would be useful by finding that the loop iterations were independent, except for the loop maintenance code.
- Use different registers to avoid unnecessary constraints that would be forced by using the same registers for different computations (e.g., name dependences).
- Eliminate the extra test and branch instructions and adjust the loop termination and iteration code.
- Determine that the loads and stores in the unrolled loop can be interchanged by observing that the loads and stores from different iterations are independent. This transformation requires analyzing the memory addresses and finding that they do not refer to the same address.
- Schedule the code, preserving any dependences needed to yield the same result as the original code.

Assembly Code of a Loop

In this section, we look at how the compiler can increase the amount of available ILP by transforming loops. This example serves both to illustrate an important technique as well as to motivate the more powerful program transformations described in Appendix H. We will rely on the following code segment, which adds a scalar to a vector:

```
for (i=999; i>=0; i=i-1)
    x[i] = x[i] + s;
```

The first step is to translate the preceding segment to RISC-V assembly language. In the following code segment, `x1` is initially the address of the element in the array with the highest address, and `f2` contains the scalar value `s`. Register `x2` is precomputed so that `Regs[x2]+8` is the address of the last element to operate on.

Instruction producing result	Instruction using result	Latency in clock cycles
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
Load double	Store double	0

Figure 3.2 Latencies of FP operations used in this chapter. The last column is the number of intervening clock cycles needed to avoid a stall. These numbers are similar to the average latencies we would see on an FP unit. The latency of a floating-point load to a store is 0 because the result of the load can be bypassed without stalling the store. We will continue to assume an integer load latency of 1 and an integer ALU operation latency of 0 (which includes ALU operation to branch).

The straightforward RISC-V code, not scheduled for the pipeline, looks like this:

```
Loop: fld    f0,0(x1)      //f0=array element
      fadd.d  f4,f0,f2     //add scalar in f2
      fsd     f4,0(x1)     //store result
      addi    x1,x1,-8     //decrement pointer
                               //8 bytes (per DW)
      bne     x1,x2,Loop   //branch x1≠x2
```

Let's start by seeing how well this loop will run when it is scheduled on a simple pipeline for RISC-V with the latencies in [Figure 3.2](#).

Scheduling of Assembly Code

The first step is to translate the preceding segment to RISC-V assembly language. In the following code segment, `x1` is initially the address of the element in the array with the highest address, and `f2` contains the scalar value `s`. Register `x2` is precomputed so that `Regs[x2]+8` is the address of the last element to operate on.

Instruction producing result	Instruction using result	Latency in clock cycles
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
Load double	Store double	0

Figure 3.2 Latencies of FP operations used in this chapter. The last column is the number of intervening clock cycles needed to avoid a stall. These numbers are similar to the average latencies we would see on an FP unit. The latency of a floating-point load to a store is 0 because the result of the load can be bypassed without stalling the store. We will continue to assume an integer load latency of 1 and an integer ALU operation latency of 0 (which includes ALU operation to branch).

Example Show how the loop would look on RISC-V, both scheduled and unscheduled, including any stalls or idle clock cycles. Schedule for delays from floating-point operations.

Answer Without any scheduling, the loop will execute as follows, taking nine cycles:

	Clock cycle issued
Loop: fld f0,0(x1)	1
stall	2
fadd.d f4,f0,f2	3
stall	4
stall	5
fsd f4,0(x1)	6
addi x1,x1,-8	7
bne x1,x2,Loop	8

We can schedule the loop to obtain only two stalls and reduce the time to seven cycles:

Loop: fld f0,0(x1)
addi x1,x1,-8
fadd.d f4,f0,f2
stall
stall
fsd f4,8(x1)
bne x1,x2,Loop

The stalls after `fadd.d` are for use by the `fsd`, and repositioning the `addi` prevents the stall after the `fld`.

Loop Unrolled Assembly Code

Example Show our loop unrolled so that there are four copies of the loop body, assuming $x1 - x2$ (that is, the size of the array) is initially a multiple of 32, which means that the number of loop iterations is a multiple of 4. Eliminate any obviously redundant computations and do not reuse any of the registers.

Answer Here is the result after merging the `addi` instructions and dropping the unnecessary `bne` operations that are duplicated during unrolling. Note that $x2$ must now be set so that $\text{Regs}[x2]+32$ is the starting address of the last four elements.

The straightforward RISC-V code, not scheduled for the pipeline, looks like this:

```
Loop: fld      f0,0(x1)      //f0=array element
      fadd.d   f4,f0,f2      //add scalar in f2
      fsd      f4,0(x1)      //store result
      addi     x1,x1,-8       //decrement pointer
                               //8 bytes (per DW)
      bne      x1,x2,Loop     //branch x1≠x2
```

Let's start by seeing how well this loop will run when it is scheduled on a simple pipeline for RISC-V with the latencies in [Figure 3.2](#).

```
Loop: fld      f0,0(x1)
      fadd.d   f4,f0,f2
      fsd      f4,0(x1)      //drop addi & bne
      fld      f6,-8(x1)
      fadd.d   f8,f6,f2
      fsd      f8,-8(x1)     //drop addi & bne
      fld      f0,-16(x1)
      fadd.d   f12,f0,f2
      fsd      f12,-16(x1)   //drop addi & bne
      fld      f14,-24(x1)
      fadd.d   f16,f14,f2
      fsd      f16,-24(x1)
      addi     x1,x1,-32
      bne      x1,x2,Loop
```

We have eliminated three branches and three decrements of $x1$. The addresses on the loads and stores have been compensated to allow the `addi` instructions on $x1$ to be merged. This optimization may seem trivial, but it is not; it requires symbolic substitution and simplification. Symbolic substitution and simplification will rearrange expressions so as to allow constants to be collapsed, allowing an expression such as $((i+1)+1)$ to be rewritten as $(i+(1+1))$ and then simplified to $(i+2)$.

Incorporating Scheduling in Unrolled Code

Example Show the unrolled loop in the previous example after it has been scheduled for the pipeline with the latencies in Figure 3.2.

```

Loop: fld    f0,0(x1)
      fadd.d  f4,f0,f2
      fsd    f4,0(x1)      //drop addi & bne
      fld    f6,-8(x1)
      -----
      fadd.d  f8,f6,f2
      fsd    f8,-8(x1)     //drop addi & bne
      fld    f0,-16(x1)
      fadd.d  f12,f0,f2
      fsd    f12,-16(x1)  //drop addi & bne
      -----
      fld    f14,-24(x1)
      fadd.d  f16,f14,f2
      fsd    f16,-24(x1)
      addi   x1,x1,-32
      bne    x1,x2,Loop
  
```

Answer

```

Loop: fld    f0,0(x1)
      fld    f6,-8(x1)
      fld    f0,-16(x1)
      fld    f14,-24(x1)
      fadd.d  f4,f0,f2
      fadd.d  f8,f6,f2
      fadd.d  f12,f0,f2
      fadd.d  f16,f14,f2
      fsd    f4,0(x1)
      fsd    f8,-8(x1)
      fsd    f12,16(x1)
      fsd    f16,8(x1)
      addi   x1,x1,-32
      bne    x1,x2,Loop
  
```

Instruction producing result	Instruction using result	Latency in clock cycles
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
Load double	Store double	0

Figure 3.2 Latencies of FP operations used in this chapter. The last column is the number of intervening clock cycles needed to avoid a stall. These numbers are similar to the average latencies we would see on an FP unit. The latency of a floating-point load to a store is 0 because the result of the load can be bypassed without stalling the store. We will continue to assume an integer load latency of 1 and an integer ALU operation latency of 0 (which includes ALU operation to branch).

The execution time of the unrolled loop has dropped to a total of 14 clock cycles, or 3.5 clock cycles per element, compared with 8 cycles per element before any unrolling or scheduling and 6.5 cycles when unrolled but not scheduled.

- Loop unrolling
 - Unroll by a factor of 4 (assume # elements is divisible by 4)
 - Eliminate unnecessary instructions

Compiler Techniques

```
Loop:      fld f0,0(x1)
           fadd.d f4,f0,f2
           fsd f4,0(x1) //drop addi & bne
           fld f6,-8(x1)
           fadd.d f8,f6,f2
           fsd f8,-8(x1) //drop addi & bne
           fld f0,-16(x1)
           fadd.d f12,f0,f2
           fsd f12,-16(x1) //drop addi & bne
           fld f14,-24(x1)
           fadd.d f16,f14,f2
           fsd f16,-24(x1)
           addi x1,x1,-32
           bne x1,x2,Loop
```

■ note: number of
live registers vs.
original loop

- Pipeline schedule the unrolled loop:

Loop: fld f0,0(x1)
 fld f6,-8(x1)
 fld f8,-16(x1)
 fld f14,-24(x1)
 fadd.d f4,f0,f2
 fadd.d f8,f6,f2
 fadd.d f12,f0,f2
 fadd.d f16,f14,f2
 fsd f4,0(x1)
 fsd f8,-8(x1)
 fsd f12,-16(x1)
 fsd f16,-24(x1)
 addi x1,x1,-32
 bne x1,x2,Loop

Compiler Techniques

■ 14 cycles
■ 3.5 cycles per element

Advanced Branch Predictors

- Static Predictor
 - Taken / No-Taken
- Basic 1-bit predictor
- Basic 2-bit predictor
 - For each branch:
 - Predict taken or not taken
 - If the prediction is wrong two consecutive times, change prediction
- Correlating predictor
 - Multiple 2-bit predictors for each branch
 - One for each possible combination of outcomes of preceding n branches
 - (m,n) predictor: behavior from last m branches to choose from 2^m n -bit predictors
- Tournament predictor
 - Combine correlating predictor with local predictor

observed dynamic behavior of a single branch in isolation. As the number of instructions in flight has increased with deeper pipelines and more issues per clock, the importance of more accurate branch prediction has grown. In this section, we examine techniques for improving dynamic prediction accuracy. This section makes extensive use of the simple 2-bit predictor covered in Section C.2, and it is critical that the reader understand the operation of that predictor before proceeding.

Correlating Branch Predictors

The 2-bit predictor schemes in Appendix C use only the recent behavior of a single branch to predict the future behavior of that branch. It may be possible to improve the prediction accuracy if we also look at the recent behavior of *other* branches rather than just the branch we are trying to predict. Consider a small code fragment from the eqntott benchmark, a member of early SPEC benchmark suites that displayed particularly bad branch prediction behavior:

Looking at Other Branch Decisions

```

if (aa==2)
    aa=0;
if (bb==2)
    bb=0;
if (aa!=bb) {

```

Here is the RISC-V code that we would typically generate for this code fragment assuming that `aa` and `bb` are assigned to registers `x1` and `x2`:

```

        addi   x3,x1,-2
        bnez   x3,L1          //branch b1  (aa!=2)
        add    x1,x0,x0       //aa=0
L1:     addi   x3,x2,-2
        bnez   x3,L2          //branch b2  (bb!=2)
        add    x2,x0,x0       //bb=0
L2:     sub    x3,x1,x2        //x3=aa-bb
        beqz   x3,L3          //branch b3  (aa==bb)

```

Let's label these branches `b1`, `b2`, and `b3`. The key observation is that the behavior of branch `b3` is correlated with the behavior of branches `b1` and `b2`. Clearly, if neither branches `b1` nor `b2` are taken (i.e., if the conditions both evaluate to true and `aa` and `bb` are both assigned 0), then `b3` will be taken, because `aa` and `bb` are clearly equal. A predictor that uses the behavior of only a single branch to predict the outcome of that branch can never capture this behavior.

Branch predictors that use the behavior of other branches to make a prediction are called *correlating predictors* or *two-level predictors*. Existing correlating predictors add information about the behavior of the most recent branches to decide how to predict a given branch. For example, a (1,2) predictor uses the behavior of the last branch to choose from among a pair of 2-bit branch predictors in predicting a particular branch. In the general case, an (m,n) predictor uses the behavior of the last m branches to choose from 2^m branch predictors, each of which is an n -bit predictor for a single branch. The attraction of this type of correlating branch predictor is that it can yield higher prediction rates than the 2-bit scheme and requires only a trivial amount of additional hardware.

Example of Correlating Predictor

The simplicity of the hardware comes from a simple observation: the global history of the most recent m branches can be recorded in an m -bit shift register, where each bit records whether the branch was taken or not taken. The branch-prediction buffer can then be indexed using a concatenation of the low-order bits from the branch address with the m -bit global history. For example, in a (2,2) buffer with 64 total entries, the 4 low-order address bits of the branch (word address) and the 2 global bits representing the behavior of the two most recently executed branches form a 6-bit index that can be used to index the 64 counters. By combining the local and global information by concatenation (or a simple hash function), we can index the predictor table with the result and get a prediction as fast as we could for the standard 2-bit predictor, as we will do very shortly.

How much better do the correlating branch predictors work when compared with the standard 2-bit scheme? To compare them fairly, we must compare predictors that use the same number of state bits. The number of bits in an (m,n) predictor is

$$2^m \times n \times \text{Number of prediction entries selected by the branch address}$$

A 2-bit predictor with no global history is simply a (0,2) predictor.

How many bits are in the (0,2) branch predictor with 4K entries? How many entries are in a (2,2) predictor with the same number of bits?

The predictor with 4K entries has

$$2^0 \times 2 \times 4K = 8K \text{ bits}$$

How many branch-selected entries are in a (2,2) predictor that has a total of 8K bits in the prediction buffer? We know that

$$2^2 \times 2 \times \text{Number of prediction entries selected by the branch} = 8K$$

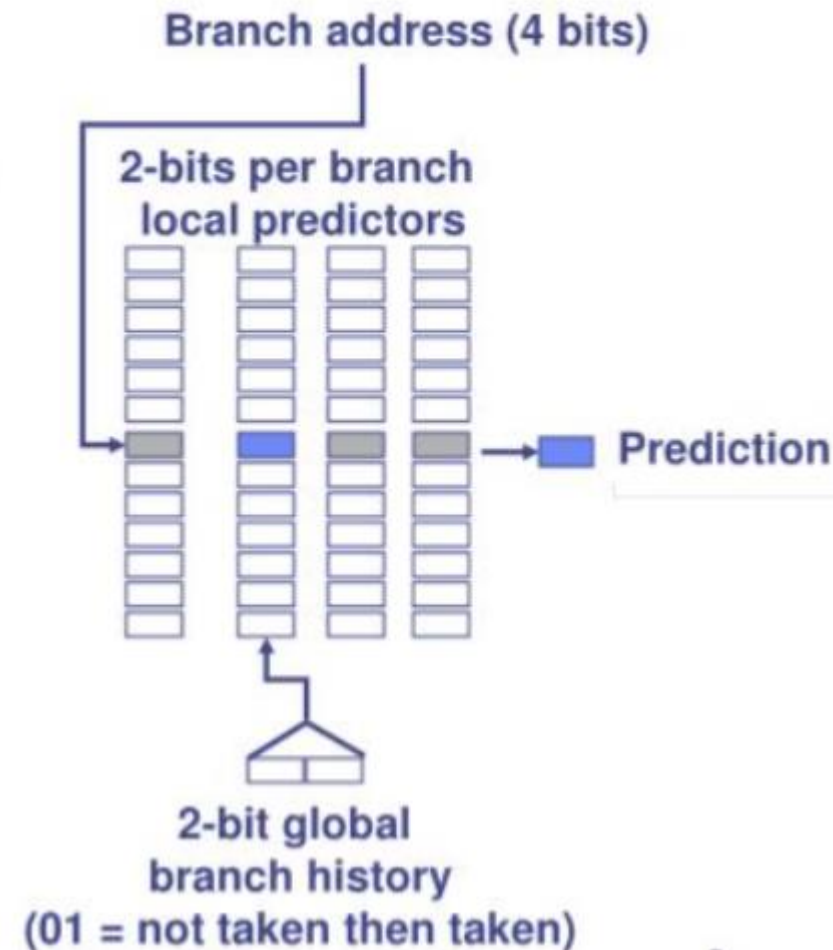
Therefore the number of prediction entries selected by the branch = 1K.

Correlating Branch Predictor

Download

General form: (m, n)
predictor

- m bits for global history, n bits for local history
- Records correlation between m+1 branches
- Simple implementation: global history can be store in a shift register
- Example: (2,2) predictor, 2-bit global, 2-bit local

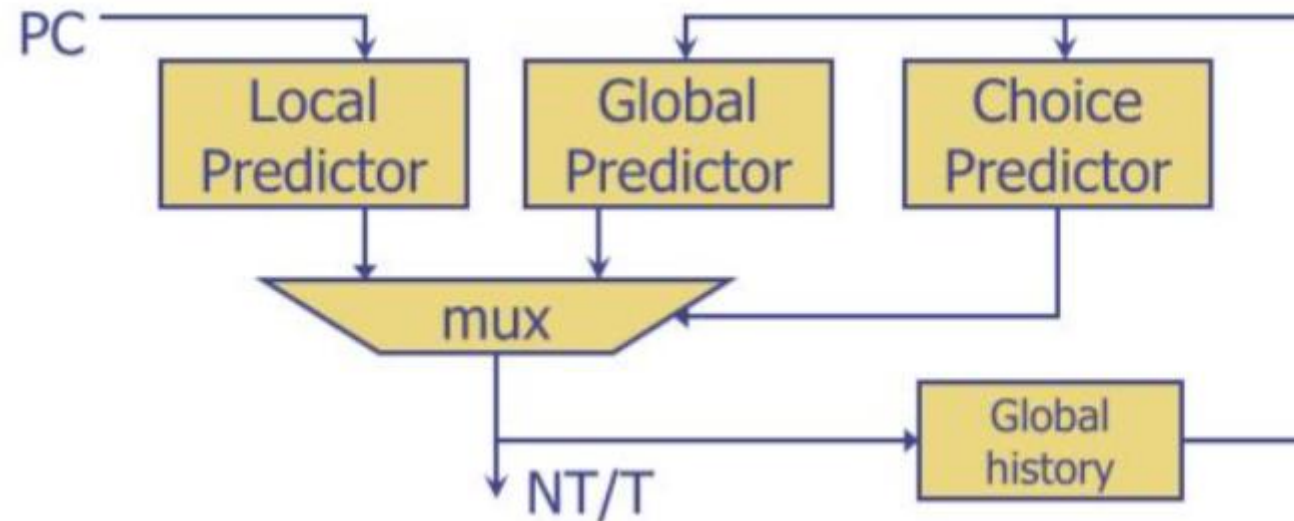


Tournament Predictors: Adaptively Combining Local and Global Predictors

The primary motivation for correlating branch predictors came from the observation that the standard 2-bit predictor, using only local information, failed on some important branches. Adding global history could help remedy this situation. *Tournament predictors* take this insight to the next level, by using multiple predictors, usually a global predictor and a local predictor, and choosing between them

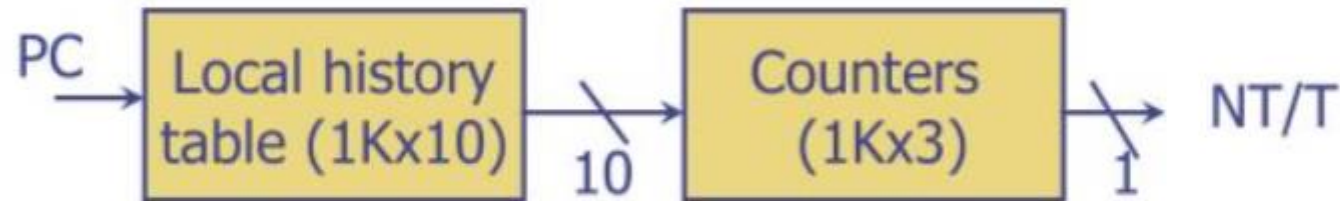
Tournament Branch Predictor

- ◆ Used in Alpha 21264: Track both “local” and global history
- ◆ Intended for mixed types of applications
- ◆ Global history: T/NT history of past k branches, e.g. 0 1 0 1 0 1 (NT T NT T NT T)

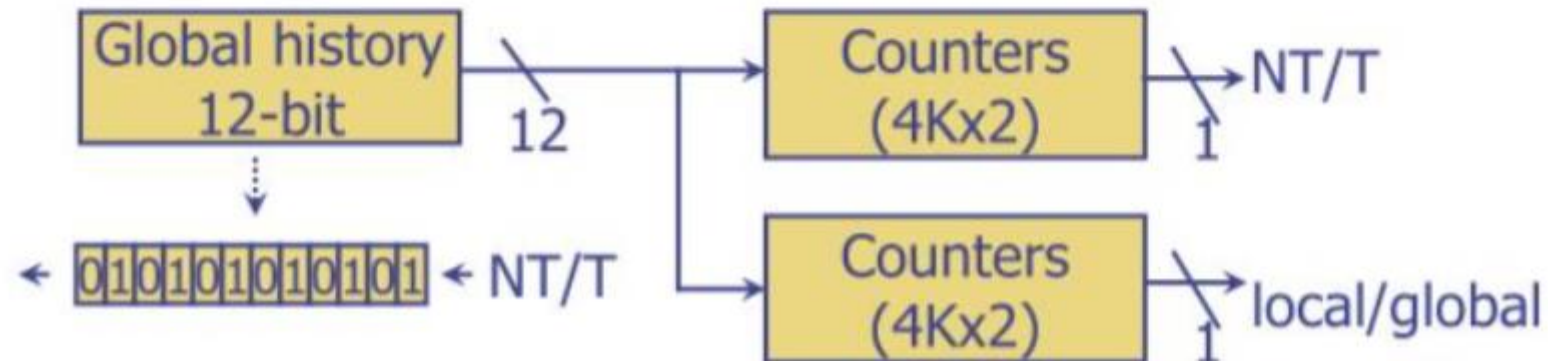


Tournament Branch Predictor

- ◆ Local predictor: use 10-bit local history, shared 3-bit counters



- ◆ Global and choice predictors:



with a selector, as shown in Figure 3.5. A *global predictor* uses the most recent branch history to index the predictor, while a *local predictor* uses the address of the branch as the index. Tournament predictors are another form of hybrid or alloyed predictors.

Tournament predictors can achieve better accuracy at medium sizes (8K–32K bits) and also effectively use very large numbers of prediction bits. Existing tournament predictors use a 2-bit saturating counter per branch to choose among two different predictors based on which predictor (local, global, or even some time-varying mix) was most effective in recent predictions. As in a simple 2-bit predictor, the saturating counter requires two mispredictions before changing the identity of the preferred predictor.

The advantage of a tournament predictor is its ability to select the right predictor for a particular branch, which is particularly crucial for the integer benchmarks.

Gshare 2-bit Predictor

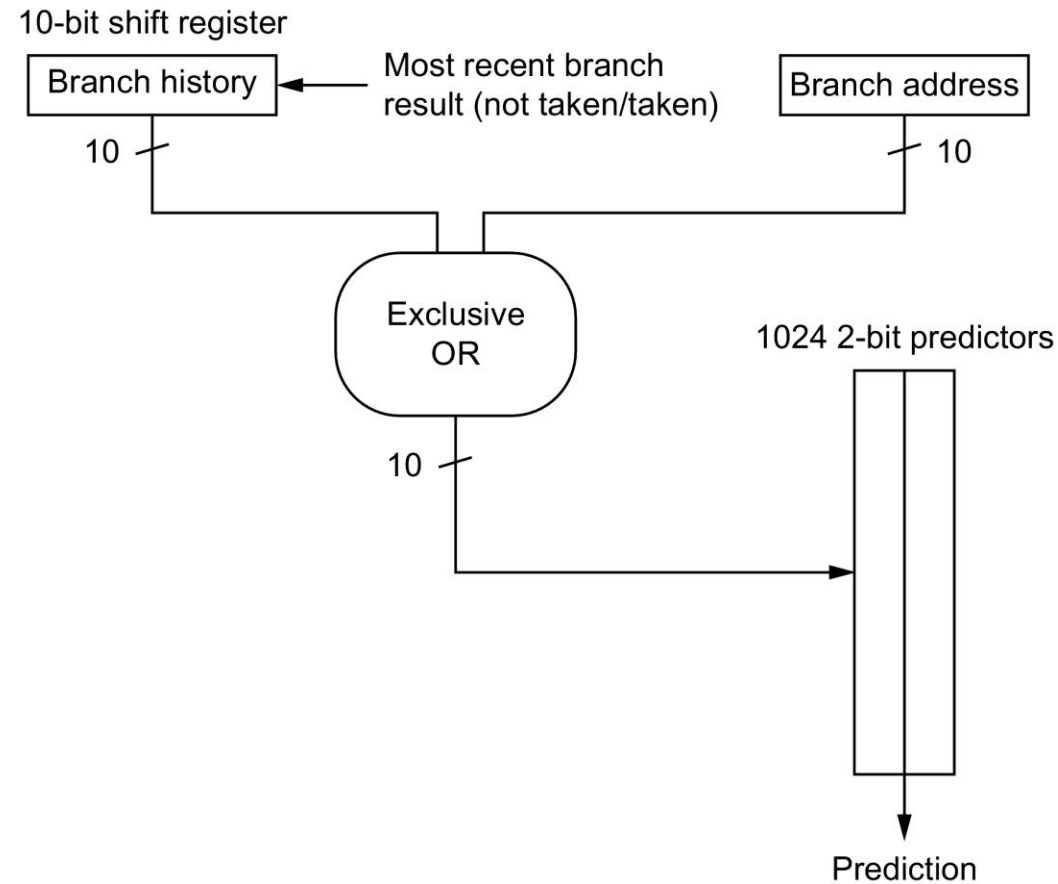


Figure 3.4 A gshare predictor with 1024 entries, each being a standard 2-bit predictor.

Tournament Global Predictor

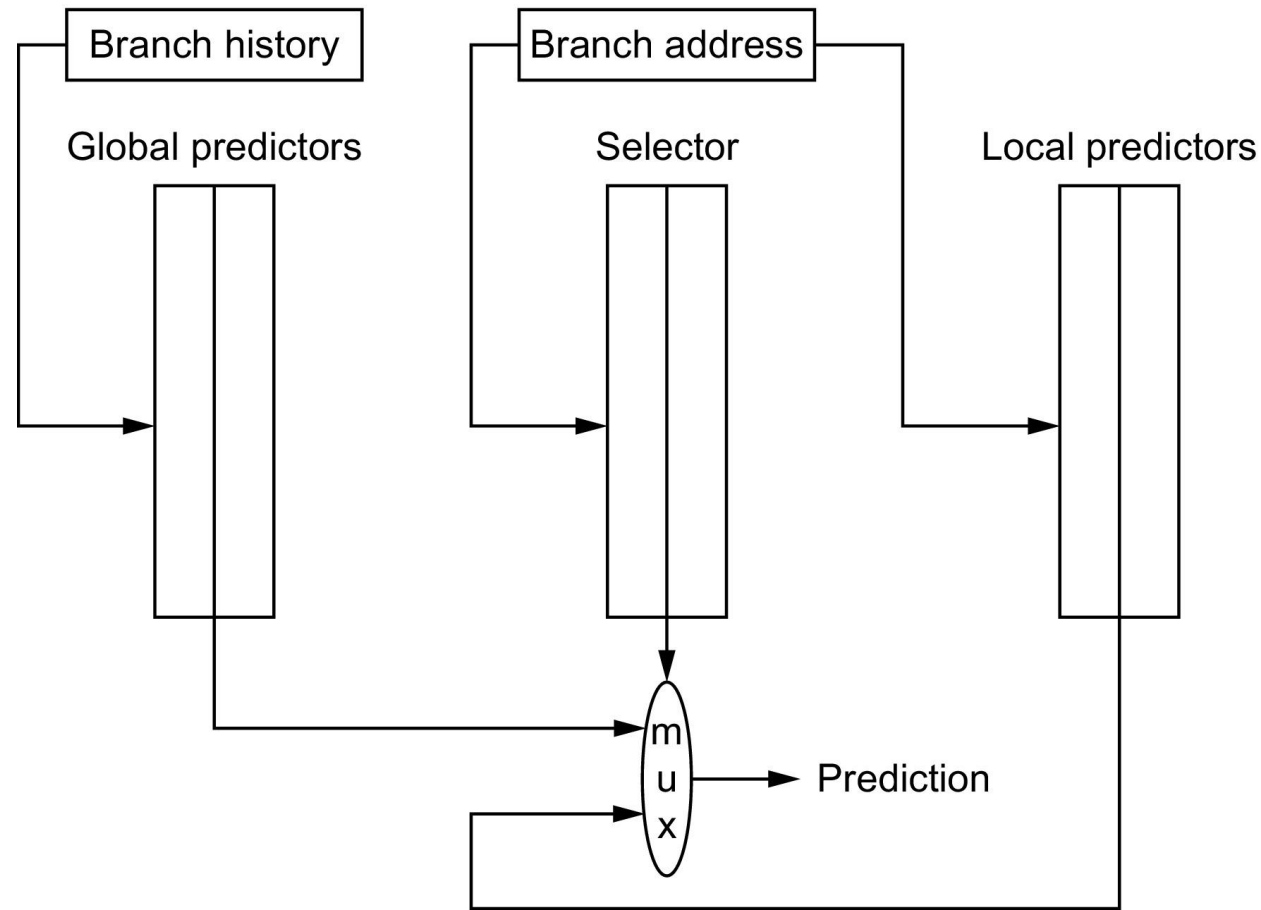
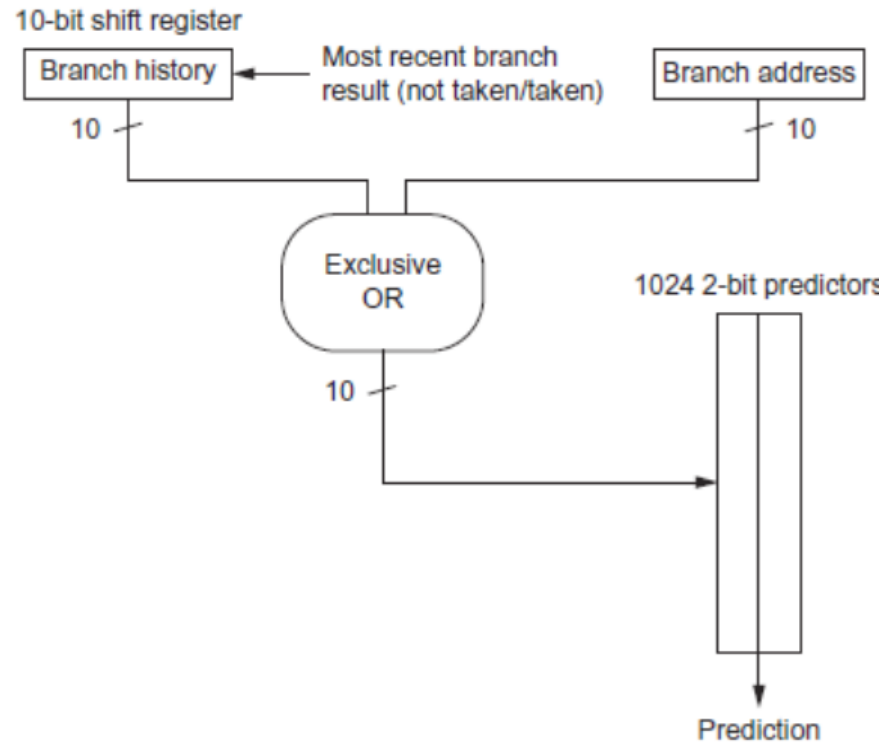
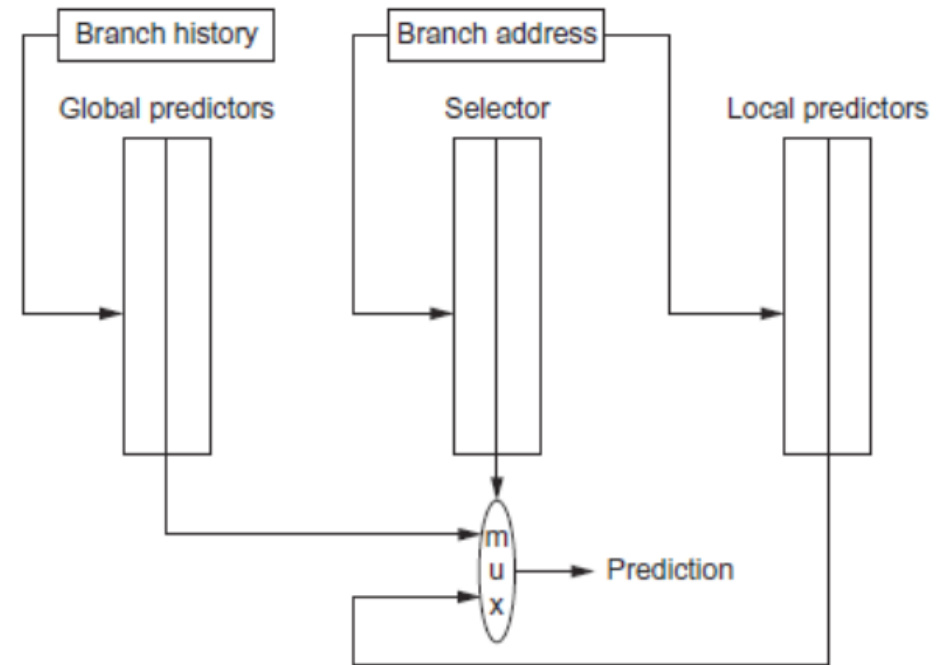


Figure 3.5 A tournament predictor using the branch address to index a set of 2-bit selection counters, which choose between a local and a global predictor. In this case, the index to the selector table is the current branch address. The two tables are also 2-bit predictors that are indexed by the global history and branch address, respectively. The selector acts like a 2-bit predictor, changing the preferred predictor for a branch address when two mispredicts occur in a row. The number of bits of the branch address used to index the selector table and the local predictor table is equal to the length of the global branch history used to index the global prediction table. Note that misprediction is a bit tricky because we need to change both the selector table and either the global or local predictor.

Diagram of Global and Tournament Predictor



Gshare 2-Bit Predictor



Tournament Global Predictor

Tagged Hybrid Predictors

The best performing branch prediction schemes as of 2017 involve combining multiple predictors that track whether a prediction is likely to be associated with the current branch. One important class of predictors is loosely based on an algorithm for statistical compression called PPM (Prediction by Partial Matching). PPM (see Jiménez and Lin, 2001), like a branch prediction algorithm, attempts to predict future behavior based on history. This class of branch predictors, which we call *tagged hybrid predictors* (see Seznec and Michaud, 2006), employs a series of global predictors indexed with different length histories.

Tagged Predictor Example

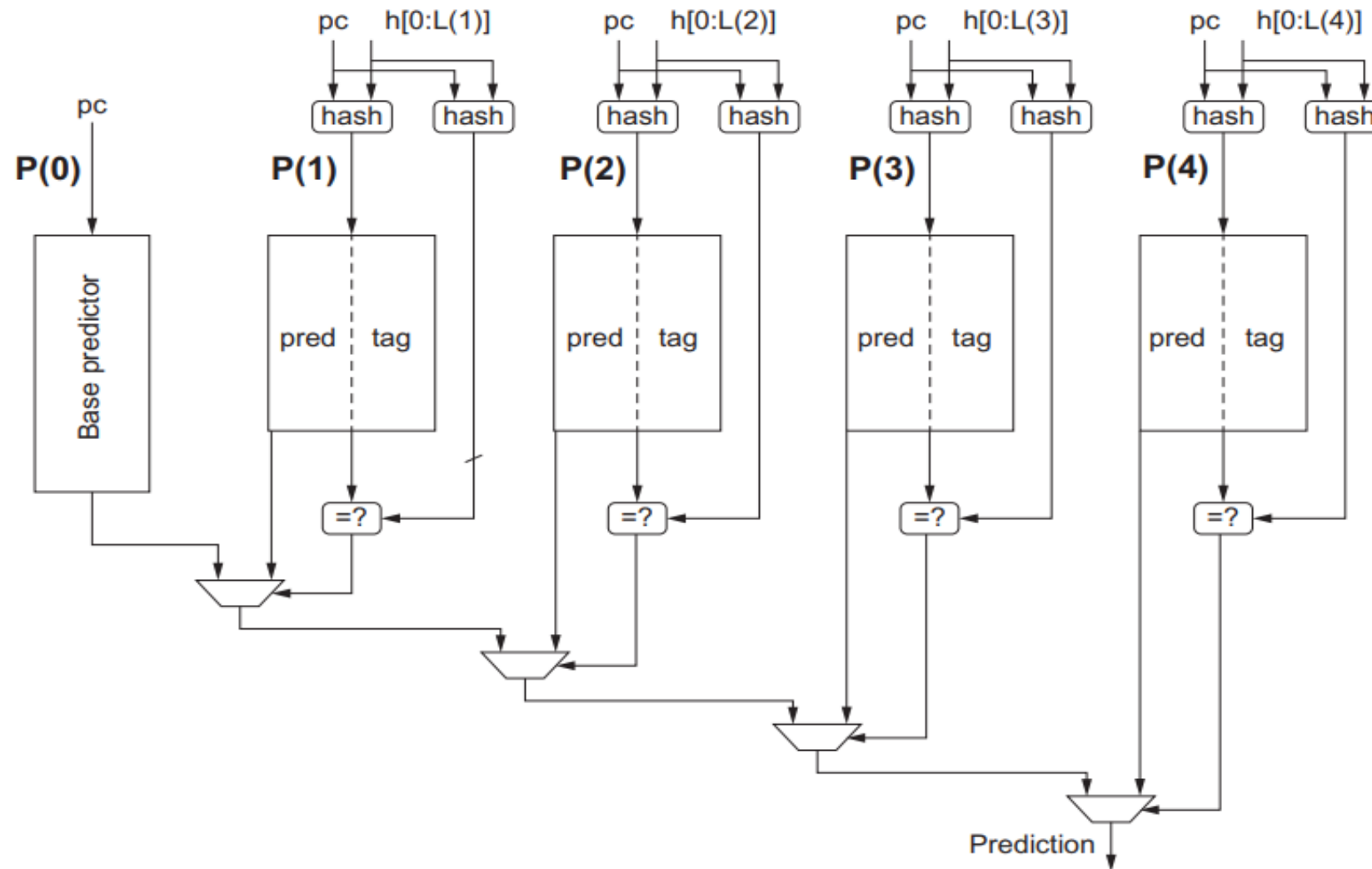


Figure 3.7 A five-component tagged hybrid predictor has five separate prediction tables, indexed by a hash of the branch address and a segment of recent branch history of length 0–4 labeled “h” in this figure. The hash can be as simple as an exclusive-OR, as in gshare. Each predictor is a 2-bit (or possibly 3-bit) predictor. The tags are typically 4–8 bits. The chosen prediction is the one with the longest history where the tags also match.

Tagged Hybrid Predictor

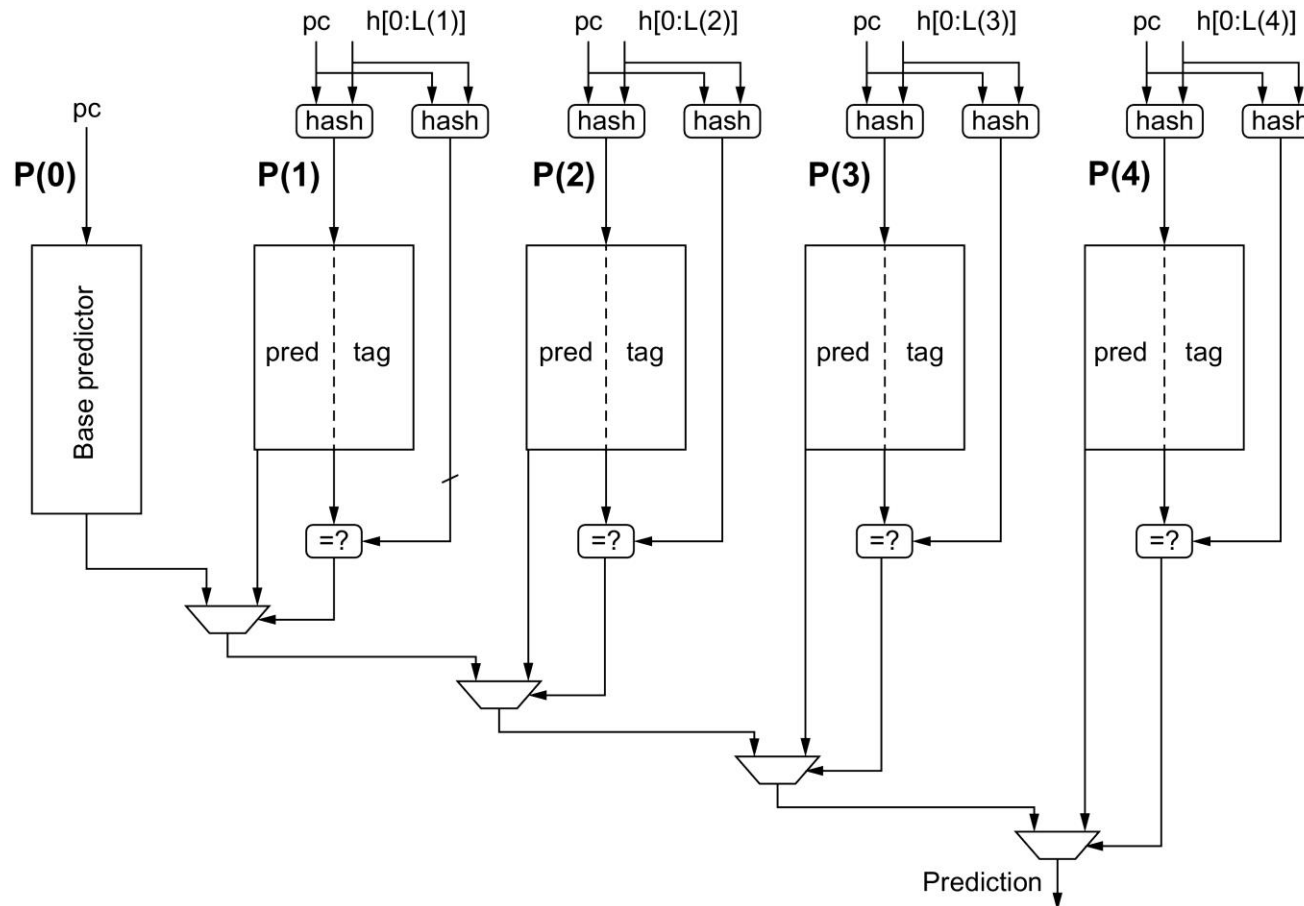


Figure 3.7 A five-component tagged hybrid predictor has five separate prediction tables, indexed by a hash of the branch address and a segment of recent branch history of length 0–4 labeled “h” in this figure. The hash can be as simple as an exclusive-OR, as in gshare. Each predictor is a 2-bit (or possibly 3-bit) predictor. The tags are typically 4–8 bits. The chosen prediction is the one with the longest history where the tags also match.

A few words about ISCA Conference



<https://www.iscaconf.org/isca2025/program/workshops.php>

https://ericrotenberg.wordpress.ncsu.edu/cbp2025/

NC STATE UNIVERSITY

Electrical and Computer Engineering →

Eric Rotenberg

About Me Teaching Research Publications Talks Students

6th Championship Branch Prediction (CBP2025)

in conjunction with:

ISCA-52 <http://iscaconf.org/isca2025/index.html>

Sponsored by ARM

The organizers of the 6th Championship Branch Prediction (CBP2025) invite you to participate in a competition for the best branch predictor.

Workshop Program

https://freddygabbay.github.io/AGPC-ISCA2025/



AGPC 2025 Workshop - Call for Papers

ARM-based General-Purpose Computing: Software-Hardware Co-Optimization for Performance Acceleration

June 21st 2025 @ Tokyo, Japan

Room: Okuma-large

Held in conjunction with [ISCA 2025](#)

About the Workshop

The International Symposium on Computer Architecture (ISCA) Workshop on "ARM General-Purpose Computing: Software-Hardware Co-Optimization for Performance Acceleration" aims to bring together researchers, engineers, and industry leaders to explore the latest advancements and innovations in leveraging ARM architecture for high-performance general-purpose computing in data center. With the growing importance of ARM-based systems in data centers and beyond, this workshop focuses on the synergy between software and hardware to achieve significant performance gains. We invite contributions that address novel algorithms, new benchmarks regarding performance and power, microarchitectural designs, hardware accelerators, tightly-coupled architectures, and optimization techniques tailored for ARM instruction set architecture. This workshop seeks to foster a

Example of Core i7 Predictor



The Core i7 920 used a two-level predictor that has a smaller first-level predictor, designed to meet the cycle constraints of predicting a branch every clock cycle, and a larger second-level predictor as a backup. Each predictor combines three different predictors: (1) the simple 2-bit predictor, which is introduced in Appendix C (and used in the preceding tournament predictor); (2) a global history predictor, like those we just saw; and (3) a loop exit predictor. The loop exit predictor uses a counter to predict the exact number of taken branches (which is the number of loop iterations) for a branch that is detected as a loop branch. For each branch, the best prediction is chosen from among the three predictors by tracking the accuracy of each prediction, like a tournament predictor. In addition to this multilevel main predictor, a separate unit predicts target addresses for indirect branches, and a stack to predict return addresses is also used.

Data Hazards and Dynamic Scheduling

Advantages of Dynamic Schedule



flow and exception behavior. Dynamic scheduling offers several advantages. First, it allows code that was compiled with one pipeline in mind to run efficiently on a different pipeline, eliminating the need to have multiple binaries and recompile for a different microarchitecture. In today's computing environment, where much of the software is from third parties and distributed in binary form, this advantage is significant. Second, it enables handling some cases when dependences are unknown at compile time; for example, they may involve a memory reference or a data-dependent branch, or they may result from a modern programming environment that uses dynamic linking or dispatching. Third, and perhaps most importantly, it allows the processor to tolerate unpredictable delays, such as cache misses, by executing other code while waiting for the miss to resolve. In Section 3.6, we explore hardware speculation, a technique with additional performance advantages, which builds on dynamic scheduling. As we will see, the advantages of dynamic scheduling are gained at the cost of a significant increase in hardware complexity.

Although a dynamically scheduled processor cannot change the data flow, it tries to avoid stalling when dependences are present. In contrast, static pipeline

The Idea behind Dynamic Scheduling



A major limitation of simple pipelining techniques is that they use in-order instruction issue and execution: instructions are issued in program order, and if an instruction is stalled in the pipeline, no later instructions can proceed. Thus, if there is a dependence between two closely spaced instructions in the pipeline, it will lead to a hazard, and a stall will result. If there are multiple functional units, these units could lie idle. If instruction j depends on a long-running instruction i , currently in execution in the pipeline, then all instructions after j must be stalled until i is finished and j can execute. For example, consider this code:

```
fdiv.d  f0,f2,f4
fadd.d  f10,f0,f8
fsub.d  f12,f8,f14
```

The `fsub.d` instruction cannot execute because the dependence of `fadd.d` on `fdiv.d` causes the pipeline to stall; yet, `fsub.d` is not data-dependent on anything in the pipeline. This hazard creates a performance limitation that can be eliminated by not requiring instructions to execute in program order.

Checking Hazards in out of order



To allow us to begin executing the `fsub.d` in the preceding example, we must separate the issue process into two parts: checking for any structural hazards and waiting for the absence of a data hazard. Thus we still use in-order instruction issue (i.e., instructions issued in program order), but we want an instruction to begin execution as soon as its data operands are available. Such a pipeline does *out-of-order execution*, which implies *out-of-order completion*.

Out-of-order execution introduces the possibility of WAR and WAW hazards, which do not exist in the five-stage integer pipeline and its logical extension to an in-order floating-point pipeline. Consider the following RISC-V floating-point code sequence:

```
fdiv.d  f0,f2,f4
fmul.d  f6,f0,f8
fadd.d  f0,f10,f14
```

There is an antidependence between the `fmul.d` and the `fadd.d` (for the register `f0`), and if the pipeline executes the `fadd.d` before the `fmul.d` (which is waiting for the `fdiv.d`), it will violate the antidependence, yielding a WAR hazard. Likewise, to avoid violating output dependences, such as the write of `f0` by `fadd.d` before `fdiv.d` completes, WAW hazards must be handled. As we will see, both these hazards are avoided by the use of register renaming.

Exceptions in out of order



Out-of-order completion also creates major complications in handling exceptions. Dynamic scheduling with out-of-order completion must preserve exception behavior in the sense that *exactly those exceptions that would arise if the program were executed in strict program order actually do arise*. Dynamically scheduled processors preserve exception behavior by delaying the notification of an associated exception until the processor knows that the instruction should be the next one completed.

Although exception behavior must be preserved, dynamically scheduled processors could generate *imprecise exceptions*. An exception is *imprecise* if the processor state when an exception is raised does not look exactly as if the instructions were executed sequentially in strict program order. Imprecise exceptions can occur because of two possibilities:

1. The pipeline may have *already completed* instructions that are *later* in program order than the instruction causing the exception.
2. The pipeline may have *not yet completed* some instructions that are *earlier* in program order than the instruction causing the exception.

Imprecise exceptions make it difficult to restart execution after an exception.

Modifications for out of order Execution

To allow out-of-order execution, we essentially split the ID pipe stage of our simple five-stage pipeline into two stages:

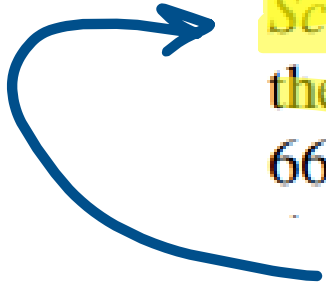
- ID
1. Issue—Decode instructions, check for structural hazards.
 2. Read operands—Wait until no data hazards, then read operands.

An instruction fetch stage precedes the issue stage and may fetch either to an instruction register or into a queue of pending instructions; instructions are then issued from the register or queue. The execution stage follows the read operands stage, just as in the five-stage pipeline. Execution may take multiple cycles, depending on the operation.

We distinguish when an instruction *begins execution* and when it *completes execution*; between the two times, the instruction is *in execution*. Our pipeline allows multiple instructions to be in execution at the same time; without this capability, a major advantage of dynamic scheduling is lost. Having multiple instructions in execution at once requires multiple functional units, pipelined functional units, or both. Because these two capabilities—pipelined functional units and multiple functional units—are essentially equivalent for the purposes of pipeline control, we will assume the processor has multiple functional units.

In Dynamic Schedule Pipeline, all instructions pass through the Issue Stage “in-order”

other in the second stage (read operands) and thus enter execution out of order. *Scoreboarding* is a technique for allowing instructions to execute out of order when there are sufficient resources and no data dependences; it is named after the CDC 6600 scoreboard, which developed this capability. Here we focus on a more sophis-

A blue handwritten arrow originates from the word 'last' in the text below and points towards the word 'Scoreboarding' in the paragraph above.

last lecture

Introduce Tomasulo



6600 scoreboard, which developed this capability. Here we focus on a more sophisticated technique, called *Tomasulo's algorithm*. The primary difference is that Tomasulo's algorithm handles antidependences and output dependences by effectively renaming the registers dynamically. Additionally, Tomasulo's algorithm can be extended to handle *speculation*, a technique to reduce the effect of control dependences by predicting the outcome of a branch, executing instructions at the predicted destination address, and taking corrective actions when the prediction was wrong. While the use of scoreboarding is probably sufficient to support simpler processors, more sophisticated, higher performance processors make use of speculation.

The IBM 360/91 floating-point unit used a sophisticated scheme to allow out-of-order execution. This scheme, invented by Robert Tomasulo, tracks when operands for instructions are available to minimize RAW hazards and introduces register renaming in hardware to minimize WAW and WAR hazards. Although there are many variations of this scheme in recent processors, they all rely on two key principles: dynamically determining when an instruction is ready to execute and renaming registers to avoid unnecessary hazards.

Dependence types⁷



- RAW (Read After Write) = “true dependence”

mul r0 * r1 -> **r2**

...

add **r2** + r3 -> r4



- WAW (Write After Write) = “output dependence”

mul r0 * r1 -> **r2**

...

add r1 + r3 -> **r2**



- WAR (Write After Read) = “anti-dependence”

mul r0 * **r1** -> r2

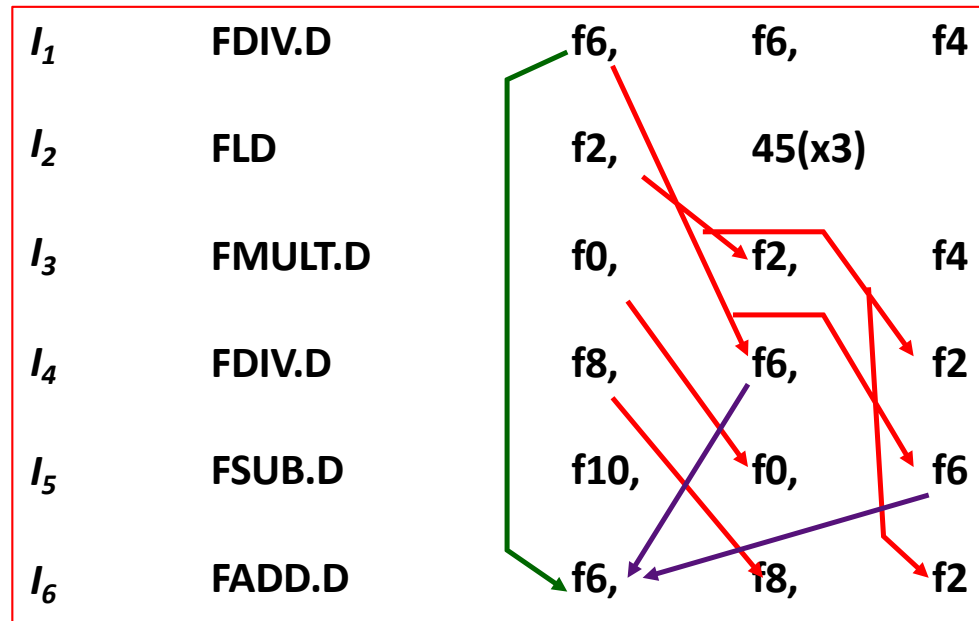
...

add r3 + r4 -> **r1**



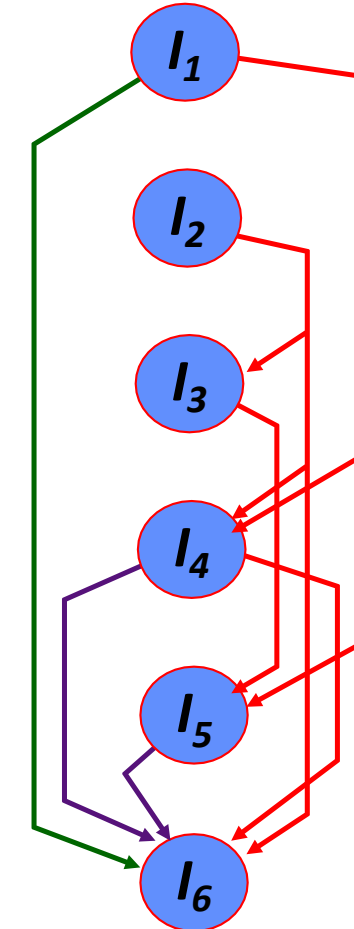
- WAW & WAR are “false”, Can be **totally eliminated** by “renaming”

Instruction Scheduling - recap



Valid orderings:

in-order	I_1	I_2	I_3	I_4	I_5	I_6
out-of-order	I_2	I_1	I_3	I_4	I_5	I_6
out-of-order	I_1	I_2	I_3	I_5	I_4	I_6



Register Renaming Summary



- **Purpose of Renaming: removing “Anti-dependencies”**
 - Get rid of WAR and WAW hazards, since these are not “real” dependencies
- **Implicit Renaming: i.e. Tomasulo**
 - Registers changed into values or response tags
 - We call this “implicit” because space in register file may or may not be used by results!
- **Explicit Renaming: more physical registers than needed by ISA.**
 - Rename table: tracks current association between architectural registers and physical registers
 - Uses a translation table to perform compiler-like transformation on the fly

RAW hazards are avoided by executing an instruction only when its operands are available, which is exactly what the simpler scoreboarding approach provides. WAR and WAW hazards, which arise from name dependences, are eliminated by register renaming. *Register renaming eliminates these hazards by renaming all destination registers, including those with a pending read or write for an earlier instruction, so that the out-of-order write does not affect any instructions that depend on an earlier value of an operand.* The compiler could typically implement such renaming, if there were enough registers available in the ISA. The original 360/91 had only four floating-point registers, and Tomasulo's algorithm was created to overcome this shortage. Whereas modern processors have 32–64 floating-point and integer registers, the number of renaming registers available in recent implementations is in the hundreds.

Register Renaming Eliminates WAR, WAW



ards, consider the following example code sequence that includes potential WAR and WAW hazards:

```
fdiv.d  f0,f2,f4
fadd.d  f6,f0,f8
fsd     f6,0(x1)
fsub.d  f8,f10,f14
fmul.d  f6,f10,f8
```

There are two antidependences: between the `fadd.d` and the `fsub.d` and between the `fsd` and the `fmul.d`. There is also an output dependence between the `fadd.d` and the `fmul.d`, leading to three possible hazards: WAR hazards on the use of `f8` by `fadd.d` and its use by the `fsub.d`, as well as a WAW hazard because the `fadd.d` may finish later than the `fmul.d`. There are also three true data dependences: between the `fdiv.d` and the `fadd.d`, between the `fsub.d` and the `fmul.d`, and between the `fadd.d` and the `fsd`.

These three name dependences can all be eliminated by register renaming. For simplicity, assume the existence of two temporary registers, `S` and `T`. Using `S` and `T`, the sequence can be rewritten without any dependences as

```
fdiv.d  f0,f2,f4
fadd.d  S,f0,f8
fsd     S,0(x1)
fsub.d  T,f10,f14
fmul.d  f6,f10,T
```

In addition, any subsequent uses of `f8` must be replaced by the register `T`. In this example, the renaming process can be done statically by the compiler. Finding any

Register Renaming for Eliminating WAW and WAW Dependencies



- Example:

DIV.D F0,F2,F4
ADD.D **F6**,F0,**F8**
S.D **F6**,0(R1)
SUB.D **T2**,F10,F14
MUL.D **T1**,F10,**T2**



DIV.D F0,F2,F4
ADD.D **F6**,F0,**F8**
S.D **F6**,0(R1)
SUB.D **F8**,F10,F14
MUL.D **F6**,F10,F8

- Now only RAW hazards remain, which can be strictly ordered

Reservation Stations for Register Renaming



In Tomasulo's scheme, register renaming is provided by reservation stations, which buffer the operands of instructions waiting to issue and are associated with the functional units. The basic idea is that a reservation station fetches and buffers an operand as soon as it is available, eliminating the need to get the operand from a register. In addition, pending instructions designate the reservation station that will provide their input. Finally, when successive writes to a register overlap in execution, only the last one is actually used to update the register. As instructions are issued, the register specifiers for pending operands are renamed to the names of the reservation station, which provides register renaming.

Because there can be more reservation stations than real registers, the technique can even eliminate hazards arising from name dependences that could not be eliminated by a compiler. As we explore the components of Tomasulo's scheme, we will return to the topic of register renaming and see exactly how the renaming occurs and how it eliminates WAR and WAW hazards.

The use of reservation stations, rather than a centralized register file, leads to two other important properties. First, hazard detection and execution control are distributed: the information held in the reservation stations at each functional unit determines when an instruction can begin execution at that unit. Second, results are passed directly to functional units from the reservation stations where they are buffered, rather than going through the registers. This bypassing is done with a common result bus that allows all units waiting for an operand to be loaded simultaneously (on the 360/91, this is called the *common data bus*, or CDB). In pipelines that issue multiple instructions per clock and also have multiple execution units, more than one result bus will be needed.

Figure 3.10 shows the basic structure of a Tomasulo-based processor, including both the floating-point unit and the load/store unit; none of the execution control tables is shown. Each reservation station holds an instruction that has been issued and is awaiting execution at a functional unit. If the operand values for that instruction have been computed, they are also stored in that entry; otherwise, the reservation station entry keeps the names of the reservation stations that will provide the operand values.

The load buffers and store buffers hold data or addresses coming from and going to memory and behave almost exactly like reservation stations, so we distinguish them only when necessary. The floating-point registers are connected by a pair of buses to the functional units and by a single bus to the store buffers. All results from the functional units and from memory are sent on the common data bus, which goes everywhere except to the load buffer. All reservation stations have tag fields, employed by the pipeline control.

- **Dynamic Scheduling of Instructions:**
 - In-order issue
 - Out-of-order execution
 - Out-of-order completion
- **Data Hazard via Register Renaming**
 - Dynamic RAW hazard detection and scheduling in data-flow fashion
 - Register renaming for WRW and WRA hazard (name conflict)
- **Implementations**
 - Scoreboard (CDC 6600 1963)
 - » Centralized register renaming
 - Tomasulo's Approach (IBM 360/91, 1966)
 - » Distributed control and renaming via reservation station, load/store buffer and common data bus (data+source)

Three Stages of Tomasulo Algorithm



1. **Issue**—get instruction from FP Op Queue

If reservation station free (no structural hazard),
control issues instr & sends operands (renames registers).

2. **Execution**—operate on operands (EX)

When both operands ready then execute;
if not ready, watch Common Data Bus for result

3. **Write result**—finish execution (WB)

Write on Common Data Bus to all awaiting units;
mark reservation station available

- Normal data bus: data + destination (“go to” bus)
- Common data bus: data + source (“come from” bus)
 - 64 bits of data + 4 bits of Functional Unit source address
 - Write if matches expected Functional Unit (produces result)
 - Does the broadcast

Three Steps in Tomasulo

let's look at the steps an instruction goes through. There are only three steps, although each one can now take an arbitrary number of clock cycles:

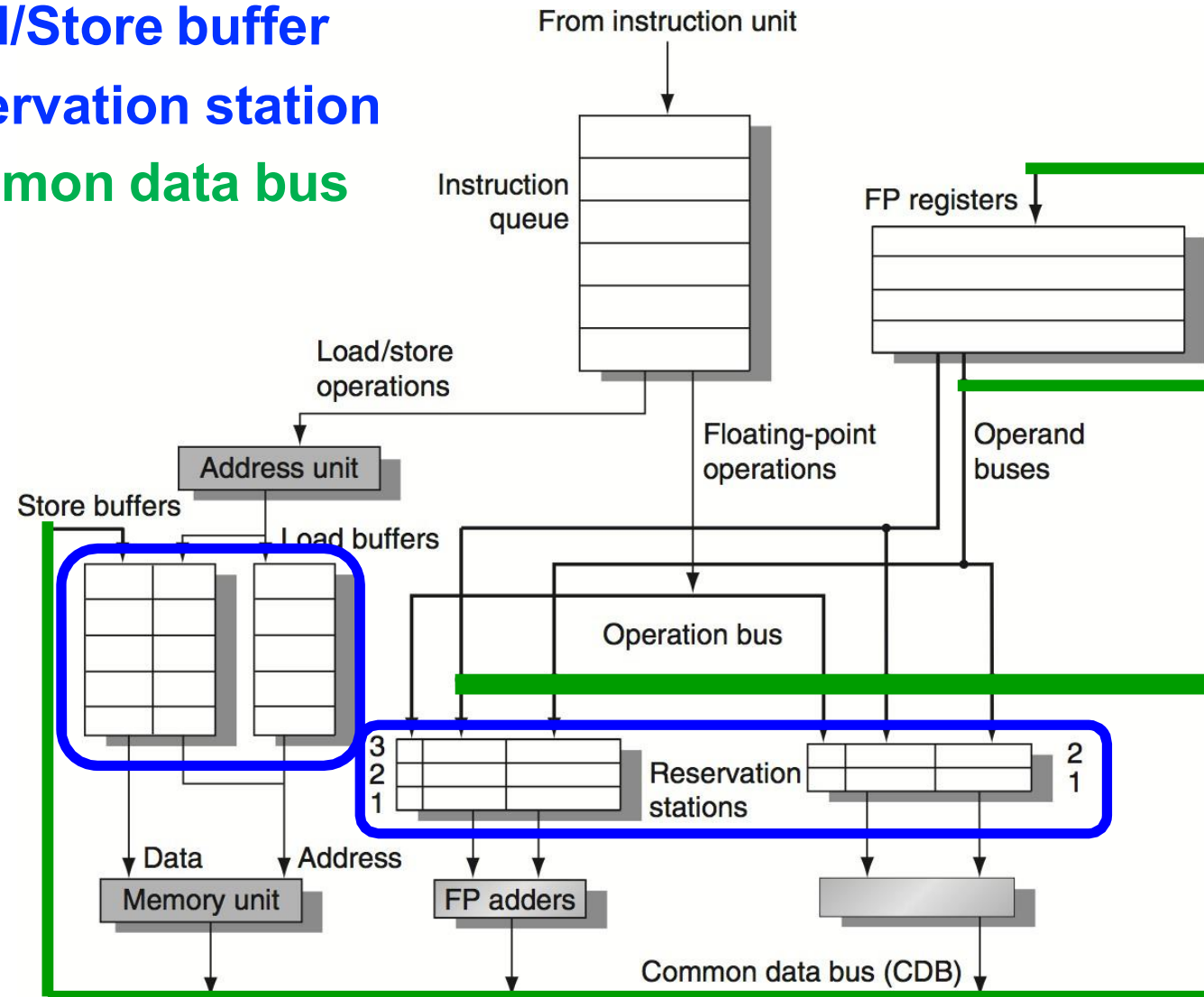
1. *Issue*—Get the next instruction from the head of the instruction queue, which is maintained in FIFO order to ensure the maintenance of correct data flow. If there is a matching reservation station that is empty, issue the instruction to the station with the operand values, if they are currently in the registers. If there is not an empty reservation station, then there is a structural hazard, and the instruction issue stalls until a station or buffer is freed. If the operands are not in the registers, keep track of the functional units that will produce the operands. This step renames registers, eliminating WAR and WAW hazards. (This stage is sometimes called *dispatch* in a dynamically scheduled processor.)
2. *Execute*—If one or more of the operands is not yet available, monitor the common data bus while waiting for it to be computed. When an operand becomes available, it is placed into any reservation station awaiting it. When all the operands are available, the operation can be executed at the corresponding functional unit. By delaying instruction execution until the operands are available, RAW hazards are avoided. (Some dynamically scheduled processors call this step “issue,” but we use the name “execute,” which was used in the first dynamically scheduled processor, the CDC 6600.)
3. *Write result*—When the result is available, write it on the CDB and from there into the registers and into any reservation stations (including store buffers) waiting for this result. Stores are buffered in the store buffer until both the value to be stored and the store address are available; then the result is written as soon as the memory unit is free.

→ Common
Data
Bus

Organizations of Tomasulo's Algorithm in RISC-V



- Load/Store buffer
- Reservation station
- Common data bus



Tomasulo Example Cycle 3

Instruction status:

Instruction		<i>j</i>	<i>k</i>	<i>Issue</i>	<i>Exec Comp</i>	<i>Write Result</i>		<i>Busy</i>	<i>Address</i>
LD	F6	34+	R2	1	3		Load1	Yes	34+R2
LD	F2	45+	R3	2			Load2	Yes	45+R3
MULTD	F0	F2	F4	3			Load3	No	
SUBD	F8	F6	F2						
DIVD	F10	F0	F6						
ADDD	F6	F8	F2						

Reservation Stations:

<i>Time</i>	<i>Name</i>	<i>Busy</i>	<i>Op</i>	<i>S1 Vj</i>	<i>S2 Vk</i>	<i>RS Qj</i>	<i>RS Qk</i>
Add1		No					
Add2		No					
Add3		No					
Mult1		Yes	MULTD		R(F4)	Load2	
Mult2		No					

Register result status:

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
3	FU	Mult1	Load2			Load1				

- Note: registers names are removed ("renamed") in Reservation Stations

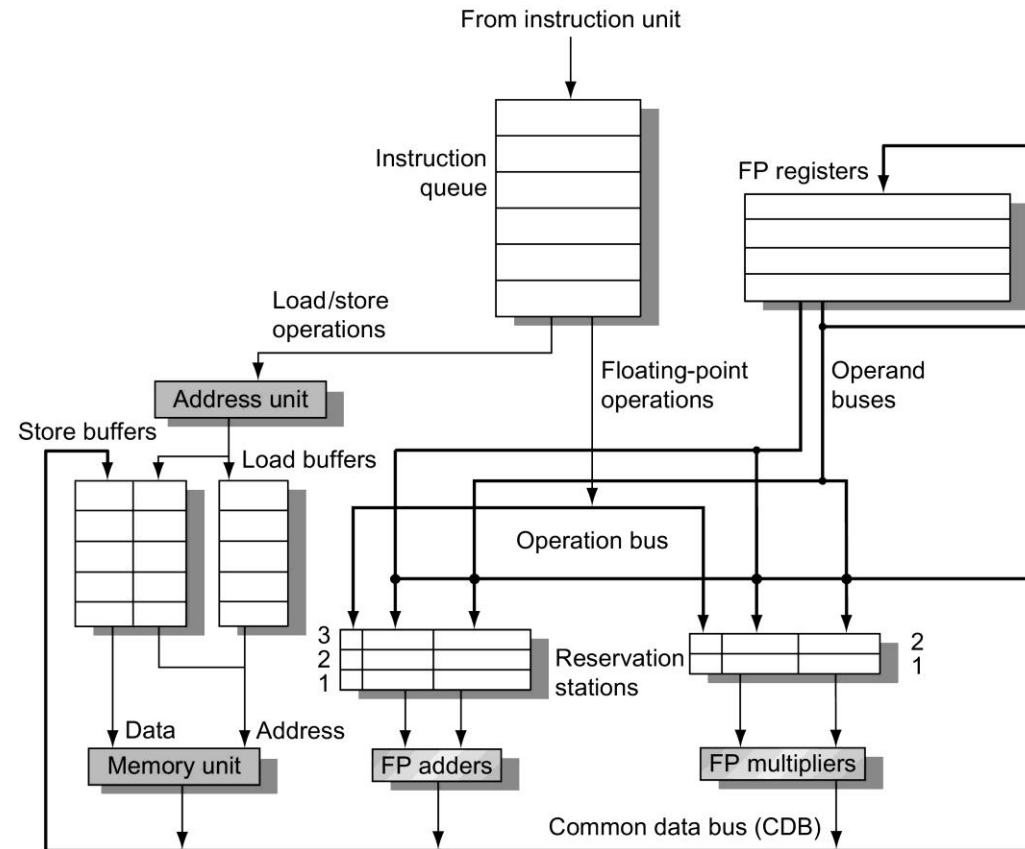


Figure 3.10 The basic structure of a RISC-V floating-point unit using Tomasulo's algorithm. Instructions are sent from the instruction unit into the instruction queue from which they are issued in first-in, first-out (FIFO) order. The reservation stations include the operation and the actual operands, as well as information used for detecting and resolving hazards. Load buffers have three functions: (1) hold the components of the effective address until it is computed, (2) track outstanding loads that are waiting on the memory, and (3) hold the results of completed loads that are waiting for the CDB. Similarly, store buffers have three functions: (1) hold the components of the effective address until it is computed, (2) hold the destination memory addresses of outstanding stores that are waiting for the data value to store, and (3) hold the address and value to store until the memory unit is available. All results from either the FP units or the load unit are put on the CDB, which goes to the FP register file as well as to the reservation stations and store buffers. The FP adders implement addition and subtraction, and the FP multipliers do multiplication and division.